

Article

Software Development and Maintenance Effort Estimation Using Function Points and Simpler Functional Measures

Luigi Lavazza ^{1,*} , Angela Locoro ²  and Roberto Meli ³ ¹ Department of Theoretical and Applied Sciences, Università degli Studi dell'Insubria, 21100 Varese, Italy² Department of Economics and Management, Università degli Studi di Brescia, 25121 Brescia, Italy; angela.locoro@unibs.it³ Data Processing Organization Srl, 00155 Roma, Italy; roberto.meli@dpo.it

* Correspondence: luigi.lavazza@uninsubria.it

Abstract: Functional size measures are widely used for estimating software development effort. After the introduction of Function Points, a few “simplified” measures have been proposed, aiming to make measurement simpler and applicable when fully detailed software specifications are not yet available. However, some practitioners believe that, when considering “complex” projects, traditional Function Point measures support more accurate estimates than simpler functional size measures, which do not account for greater-than-average complexity. In this paper, we aim to produce evidence that confirms or disproves such a belief via an empirical study that separately analyzes projects that involved developments from scratch and extensions and modifications of existing software. Our analysis shows that there is no evidence that traditional Function Points are generally better at estimating more complex projects than simpler measures, although some differences appear in specific conditions. Another result of this study is that functional size metrics—both traditional and simplified—do not seem to effectively account for software complexity, as estimation accuracy decreases with increasing complexity, regardless of the functional size metric used. To improve effort estimation, researchers should look for a way of measuring software complexity that can be used in effort models together with (traditional or simplified) functional size measures.

Keywords: unadjusted Function Points (UFPs); simple Function Points (SFPs); effort estimation; simple functional size measures



Citation: Lavazza, L.; Locoro, A.; Meli, R. Software Development and Maintenance Effort Estimation Using Function Points and Simpler Functional Measures. *Software* **2024**, *3*, 442–472. <https://doi.org/10.3390/software3040022>

Academic Editors: Alessio Ferrari and Hongyu Zhang

Received: 6 September 2024

Revised: 25 October 2024

Accepted: 26 October 2024

Published: 29 October 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Functional size measures are widely used for estimating the development effort of software, mainly because they can be obtained in the early stages of development, when effort estimates are most needed. Function Point analysis (FPA) was introduced to yield a measure of software size based exclusively on logical specifications [1].

After the introduction of the original Function Points (FPs), a few “simplified” measures have been proposed, aiming to make measurement simpler and quicker, but also to make measures applicable when fully detailed software specifications are not yet available. Among the simplified measures are simple Function Points (SFPs) [2] (formerly known as SiFPs [3]).

Following the ISO [4], we consider only unadjusted FPs (UFPs): It has been shown [5–7] that, in general, software size measures expressed in UFPs do not support more accurate effort estimation with respect to simplified measures. However, some practitioners who use UFPs for estimation believe that, when considering “complex” projects, i.e., projects that involve many complex transactions and data, UFP measures support more accurate estimates than SFP or other measures that do not account for greater-than-average complexity (throughout this paper, the notion of “complexity” used is the one supported by Function Point analysis [1], i.e., the criterion used to weight transactions and logical data

files; see also the discussion in Section 4.2.4). Previous studies did not specifically address the effect of complexity on the accuracy of effort estimation; hence, they cannot be used to confirm or disprove the aforementioned hypothesis. For this purpose, we devised and executed an empirical study, as illustrated in the rest of this paper. This study is based on the analysis of the ISBSG dataset [8], which has been widely used for studies concerning software functional size.

The simplified functional size metrics used in this study are the already mentioned SFPs and the transactional part of SFPs (tSFPs), which is equivalent to the number of transactions (or elementary processes) described in the software specifications.

This paper presents an extension of previously published results [9] that concerned uniquely new development projects. Here, we consider two additional types of projects: extensions and enhancements. Extension projects just add functionality to existing software, without changing the existing code; instead, enhancements involve additions as well as changes. Considering these two additional types of projects widens the scope of application and the type of comparisons, thus covering all kinds of software projects.

The results of this study will likely be helpful for the numerous software development organizations that use FPA, e.g., organizations that develop software for public administration and are thus required to provide software size measured via IFPUG (International Function Point User Group) FPA by local laws (as in Brazil, Italy, Japan, South Korea, and Malaysia). Also, other organizations may need FP measures because they use effort estimation tools (like Galorath's Seer-SEM [10], for instance) that take the size expressed in FPs as input (together with several parameters that account for the development process and technology, non-functional requirements, human factors, etc.).

The results of this study can be interesting also for organizations that use agile development processes. In fact, traditional functional size measurement is not very popular in agile contexts because it is perceived as a "heavy" method not suitable for agile development. Specifically, agile requirements are considered too light and inconsistent to be exploited with the above method; moreover, traditional size measurement methodologies are perceived as an imposition, thus lacking acceptability in the agile domain [11]. Instead, simplified functional size measurement methods could fit easily in agile development practices, especially when the simplification is pushed to considering only transactions, which are functional elements that can be easily identified from user stories.

This paper is organized as follows. Section 2 recalls some basic notions concerning functional size measurement methods. Section 3 states the objectives of the work described here, also by formulating research questions. Section 4 describes the empirical study through which we addressed the research questions; the achieved results are also illustrated. In Section 5, research questions are answered. Section 6 discusses the threats to the validity of the study. Section 7 accounts for related work. Finally, Section 8 draws some conclusions and outlines future work.

2. Background

In this section, we provide a very brief introduction to Function Points, as well as to simplified measures, namely SFPs and their transactional component.

2.1. Function Point Analysis

Function Point analysis was originally introduced by Albrecht to measure the size of software systems from end-users' point of view, with the goal of estimating the development effort [1]. Currently, FPA is officially documented by the IFPUG (International Function Points User Group) via the counting practices manual [12].

The basic idea of FPA is that the "amount of functionality" released to the user can be evaluated by taking into account (1) the data used by the application to provide the required functions and (2) the elementary processes or transactions (i.e., operations that involve data crossing the boundaries of the application) through which the functionality is delivered to the user. Both data and transactions are evaluated at the conceptual level, i.e.,

they represent data and operations that are relevant to the user. Therefore, IFPUG Function Points are counted on the basis of functional user requirement specifications.

Functional user requirements are modeled as a set of base functional components: the size of the application is obtained as the sum of the sizes of base functional components. Functional components are data functions (also known as logical files), which are classified into internal logical files (ILFs) and external interface files (EIFs), and transactional functions, which are classified into external inputs (EIs), external outputs (EOs), and external inquiries (EQs), according to the activities carried out within the considered process and their primary intent. The size of every base functional component is determined by its type and its “complexity” (see the manual [12] for details). The functional size of a given application, expressed in unadjusted Function Points, is given by the sum of the sizes of all its base functional components.

Function Point Analysis also involves the “adjustment” in the size measured in UFPs to obtain a value that is expected to be better correlated to development effort. However, the International Standardization Organization (ISO) only allowed for unadjusted Function Points as a functional size measure [4]. In accordance with ISO, in this paper, we consider only UFPs.

The core of FPA involves the following main activities:

1. Identifying data functions.
2. Identifying transactional functions.
3. Classifying data functions as ILFs or EIFs.
4. Classifying transactional functions as EIs, EOs, or EQs.
5. Determining the complexity of each data function.
6. Determining the complexity of each transactional function.

The first four of these activities can be carried out even if the functional user requirements have not yet been fully detailed. On the contrary, the last two activities require that details are available.

Simplified functional size measurement methods aim to provide estimates of functional size measures by skipping one or more of the activities listed above. Specifically, simplified measurement methods tend to skip at least the determination of complexity, since this activity is time- and effort-consuming [13].

2.2. Simple Function Points

The simple Function Point (SiFP) measurement method [2,3] has been designed by Meli to be lightweight and easy to use. Later on, IFPUG acquired the SiFP rights and developed the IFPUG SFP method, maintaining the original structure but incorporating the terminology of the original FPA method.

Like IFPUG FPA, the SFP method is independent of the technologies and of the technical design principles. It requires only the identification of elementary processes (EPs) and logical files (LFs) based on the assumption that value to an EP or LF is given as a whole, independently of the internal organization and details. Note that both EPs and LFs are concepts defined in traditional FPA: in practice, elementary processes are transactions (ignoring whether they are inputs, outputs, or inquiries) and logic files are data (ignoring whether they are internal or external). Therefore, SFP measurement only requires carrying out steps 1 and 2 of the procedure described in Section 2.1 above.

SFP assigns a numeric value directly to EPs and LFs as follows:

$$Size_{SFP} = 7 \#LF + 4.6 \#EP$$

thus speeding up the functional sizing process at the expense of ignoring the domain data model and the primary intent of each elementary process.

The weights for EPs and LFs were originally defined to achieve the best possible approximation of FPA. However, since the SFP is a measurement method, those weights

are constants, i.e., they are not subject to update or change for approximation reasons, and are now crystallized for stability, repeatability, and comparability reasons.

2.3. Even More Simplified Functional Size Measures

As described in Section 2.2 above, the measure of SFPs considers both elementary processes and logical data files. A further simplification consists of not considering data at all in the measurement of functional size. Accordingly, in this paper we also evaluate the transactional component for SFPs (denoted as tSFPs) as a further simplified measure of functional size that can be used for effort estimation.

Since $tSFP = 4.6 \#EP$, considering only the transactional component of SFPs equates to considering only the number of transactions.

3. Research Questions

Some research has already been dedicated to evaluating the possibility of using functional size measures that are definitely simpler than standard IFPUG UFPs for effort estimation [5,6]. Simpler metrics are of great interest for practitioners because they are quicker and less expensive to collect than traditional FPs, and, even more importantly, simple measures can sometimes be applied before detailed and complete software requirements are available.

However, previous research proposed empirical studies whose conclusions were based on the evaluation of estimation accuracy over the entire test set. Such practice, although sound and informative, does not solve possible doubts about the performance of different metrics when dealing with projects having different complexity.

In fact, in some environments, it is believed that traditional UFPs are better at accounting for the complexity of projects; hence, when dealing with relatively complex projects, UFPs are expected to support more accurate effort estimation with respect to simpler functional size measurement methods. However, as far as we know, hardly any evidence has been produced to support this belief (except, in part, our previous conference paper [9]).

Note that, in this paper, by “complexity” we refer to the notion of complexity as defined in Function Point analysis. Therefore, the complexity of a transaction depends on the amount of input/output data and the number of logic data files involved in the execution of the transaction. Other notions of complexity (such as McCabe’s, for instance) are not considered since they do not contribute to functional size as defined by standards [14].

In this paper, we provide some evidence that can be used to either support or disprove the aforementioned belief. To this end, we formulate the following research questions:

- RQ1** If project complexity is not taken into account, is it true that simple functional measures (namely, SFPs and tSFPs) provide effort estimates that are as accurate as those provided by IFPUG UFPs?
- RQ2** For projects that have relatively high (respectively, low) complexity, do UFPs and simple functional metrics (namely, SFPs and tSFPs) support effort estimation at significantly different levels of accuracy?

As mentioned in the introduction, we consider three types of projects: new developments, extensions, and enhancements. It is reasonable to expect that the same functional size is associated with different amounts of effort, depending on whether software is developed from scratch, added to existing code, or if the activity involves a mix of additions, changes, and deletions. Therefore, RQ1 and RQ2 are applied to each one of the three aforementioned types of projects. In what follows, we use the labels NEW, EXT, and ENH to denote new developments, extensions, and enhancements, respectively.

It is well known that there are multiple ways for (i) modeling the dependence of development effort on software functional size; (ii) evaluating (in a statistically sound manner) the accuracy of the obtained estimates; (iii) classifying projects as relatively complex or relatively simple, etc. Answering the research questions for all the possible ways of addressing the issues mentioned above is hardly possible. Therefore, in this paper, we

adopt reasonable models and classification techniques, preferring simpler ones, to avoid the risk of obtaining results that depend on the intricacies of the technical instruments being used.

4. The Study

In this section, we describe the empirical study that supports our answers to the research questions. In Sections 4.3–4.5, the raw results for each project type (new development, extension, and enhancement projects) are reported for all metrics (UFPs, SFPs, and tSFPs). The answers to each of the RQs, for all project types and comparisons (UFPs vs. SFPs and UFPs vs. tSFPs), are reported in Section 5.

4.1. The Dataset

In our empirical study, we analyzed data from the ISBSG dataset [8], which includes data from real-life software development projects and has been widely used in studies involving Functional Size measures.

To perform the analysis described in this paper, we needed more detailed information than that present in “regular” versions of the ISBSG dataset. For instance, the versions of the ISBSG dataset that are usually released to the public provide the functional size of each project split into the size of EIs, EOs, EQs, ILFs, and EIFs, but do not specify how many EIs (or EOs, EQs, etc.) have high, mid, or low complexity. Similarly, the regular ISBSG dataset indicates the size of added functionality, but does not specify how much of the added functionality is due to added EIs, how much to added EOs, etc. Luckily, the ISBSG organization collects more data than they include in the versions of the datasets that are released to users. Therefore, we asked the ISBSG for a view of their internally managed data that included the data that we needed. This custom view includes fewer records than the commercially released versions; namely, it contains data from 1307 projects, while the “regular” ISBSG dataset includes several thousand records.

Among the data that characterize each project are the “Data quality rating” (concerning the completeness and reliability of the data) and “UFP rating” (concerning the trustworthiness of the UFP counting). Both are graded “A” (best) to “D” (worst), and ISBSG itself suggests using only data rated “A” or “B”. Following a consolidated practice [15], we used only the highest-quality records, i.e., those rated “A” or “B”.

The dataset contains data from both projects addressing the development of new software products and projects addressing the enhancement of existing projects. Based on the available data, we were able to further split enhancements (as classified in the dataset) into proper extensions (i.e., projects that add functionalities without changing the existing ones) and proper enhancements (i.e., projects that involve changing or deleting some of the existing functionalities).

For each project, many measures are provided. Of these, we used the following:

- The effort spent, expressed in PersonHours (PHs).
- The size, expressed in IFPUG Function Points.
- #ILF, #EIF, #EI, #EO, and #EQ (i.e., the number of ILFs, EIFs, EIs, EOs, and EQs), each split per complexity (high, medium, low) and activity type (added, changed, deleted).

The considered version of the ISBSG dataset contains some measures (namely, effort, the size in UFPs, and the number of transactions) that we used as-is, as well as raw data (#ILF, #EIF, #EI, #EO, and #EQ) that we used to compute $\#EP = \#EI + \#EO + \#EQ$ and $\#LF = \#ILF + \#EIF$; hence, $SFP = 4.6\#EP + 7\#LF$ (and $tSFP = 4.6\#EP$). In this respect, it is worth noting that we obtained $\#EP$ as the sum of $\#EI$, $\#EO$, and $\#EQ$, and $\#LF$ as the sum of $\#ILF$ and $\#EIF$ because the data in the dataset were originally collected for computing Function Points. A measurer that analyzes functional user requirements with the purpose of computing SFPs would not classify transactions as EIs, EOs, and EQs or data files as ILFs or EIFs; hence, they would have directly obtained $\#EP$ and $\#LF$.

The dataset includes data from 533 new development, 128 extension and 646 enhancement projects. Descriptive statistics of the ISBSG dataset are given in Sections 4.3–4.5.

4.2. The Method

4.2.1. The Effort Model

In this paper, we use a very simple method for building effort models. In fact, we assume that effort can be computed by dividing the size of the software product by the observed productivity:

$$Effort = \frac{Size}{Productivity} \quad (1)$$

It is clear that Formula (1) describes a very simple model of effort, since (i) it assumes that effort depends only on functional size, and (ii) it is structurally simple, especially when compared with models that can be obtained via sophisticated techniques like machine learning, neural networks, etc. We preferred this extremely simple model to avoid possible confounding effects, being exclusively interested in the role played by size in determining development effort.

Productivity is defined as [16]

$$Productivity = \frac{Size}{Effort} \quad (2)$$

However, the value of *Productivity* to be used in (1) can be obtained in different ways. In this paper, we consider two possible derivations of the *Productivity* value:

1. For each project in the dataset, we considered its *Productivity*, as defined in (2). Then we computed the mean value of the projects' productivity. In performing this, we used as a size measure UFPs, SFPs, and tSFPs, thus obtaining $Productivity_{UFP}$, $Productivity_{SFP}$ and $Productivity_{tSFP}$.
2. We proceeded as described above, but the productivity was obtained as the median value of the projects' productivity.

Productivity was then used to compute, via Formula (1), the estimated effort for each project in the dataset. The process was repeated separately for NEW, EXT, and ENH projects. Then, we computed estimation errors: for the i th project, the estimation error $EstErr_i$ is

$$EstErr_i = ActualEffort_i - EstimatedEffort_i = ActualEffort_i - \frac{Size_i}{Productivity} \quad (3)$$

Specifically, the computation described by Formula (3) was computed for the three considered functional size measures, i.e., UFPs, SFPs, and tSFPs. For instance,

$$EstErr_{i,UFP} = ActualEffort_i - \frac{Size_{i,UFP}}{Productivity_{UFP}}$$

where $Size_{i,UFP}$ is the functional size of the i th project, expressed in UFPs. Similarly, we obtained $EstErr_{i,SFP}$ and $EstErr_{i,tSFP}$ for each project in the ISBSG dataset.

4.2.2. Evaluation of Estimation Accuracy

We performed a sign test to evaluate whether any of the considered measures supports more accurate effort estimates than the other considered functional size measurement methods. For instance, we counted for how many projects UFPs are a better effort predictor than SFPs: let n_{UFP} be that number; similarly, we counted for how many projects SFPs are a better effort predictor than UFPs: let n_{SFP} be that number. Using the binomial test (with $\alpha = 0.05$), we evaluated whether we can safely conclude that estimates based on UFPs are more accurate than estimates based on SFPs. In practice, we tested if the probability that $n_{UFP} > n_{SFP}$ is greater than $\frac{1}{2}$.

In the process described above, we had to consider that it is possible that two size measures obtain extremely similar, though different, estimation errors. This situation can be quite misleading. Consider, for instance, a situation where, in 90% of the cases, it is $|EstErr_{i,X}| = |EstErr_{i,Y}| + 1$, and, in 10% of the cases, it is $|EstErr_{i,X}| = 2 |EstErr_{i,Y}|$, where

X and Y are two size measures. In this example, using Y would be preferable because it yields definitely better estimates in 10% of the cases while, in the remaining cases, the estimation error is practically the same (being 1 PH error negligible). However, the sign test based on the consideration that, in 90% of the cases, it is $|EstErr_{i,X}| < |EstErr_{i,Y}|$, would conclude that X is the best predictor. Therefore, we consider the estimation errors equivalent when $|EstErr_{i,X} - EstErr_{i,Y}| < 0.01 \cdot ActualEffort$, that is, when the magnitude of the error difference is not greater than 1% of the actual effort.

For each estimation, we have that measure X yields better estimates with respect to measure Y p times, and equivalent and worse estimates, respectively, e and n times. Based on these numbers, we propose the following evaluations:

- X and Y are equally accurate if the binomial tests involving p and n do not reject the null hypothesis that $p = n$ and reject the null hypothesis that $p > n$ or $p < n$. This situation is represented with the symbol “=” in the tables below.
- X is more accurate than Y if the binomial test rejects the null hypothesis that $p \leq e + n$. This situation is represented with the symbol “>”.
- X is less accurate than Y if the binomial test rejects the null hypothesis that $p \geq n$. This situation is represented with the symbol “<”. Note that $p < n$ implies that $p < e + n$.
- The remaining cases occur when $p > n$, but there is no statistically significant evidence that $p > e + n$, and when $p < e + n$, but there is no statistically significant evidence that $p < n$. These cases are represented with symbols “ $\sim$ >” and “ $\sim$ <”, respectively.

Based on these rules, let us consider the following examples:

Example 1. $p = 117$, $e = 20$, and $n = 41$. In this case, we obtain a clear response: X is preferable to Y . In fact, the collected evidence supports the hypothesis that X achieves more accurate estimates for the majority of the projects. On the contrary, Y achieves more accurate estimates for a minority of the projects, even excluding those (20 projects) estimated equally well by X and Y . In the following tables, this situation is represented by “ $X > Y$ ” and “ $Y < X$ ”.

Example 2. $p = 59$, $e = 23$, and $n = 97$. In this case, we do not obtain a conclusive indication. The collected evidence shows that Y seems better than X , but does not support the hypothesis that Y achieves more accurate estimates for the majority of the projects. At any rate, there is evidence that X achieves more accurate estimates for a minority of the projects (namely, when excluding equivalent cases). In the following tables, this situation is represented by “ $X < Y$ ” and “ $Y \sim > X$ ”.

The evaluations described above are complemented by the computation of the estimation errors, which are represented via boxplots and also evaluated via the mean absolute residual (MAR) and the median absolute residual (MdAR).

MAR—also known as the mean absolute error (MAE)—is an unbiased indicator, recommended by several authors (e.g., [17]). It is computed as the mean of absolute estimation errors: $MAR = \frac{1}{n} \sum_{i=1}^n |EstErr_i|$. MdAR is the median of the absolute errors.

4.2.3. Classification of Projects According to Complexity

Research question RQ2 requires identifying projects that are “complex”. To this end, we need to properly define the notion of complexity. In the context of Function Point analysis, complexity is evaluated by weighting base functional components. Therefore, we followed this practice to evaluate projects’ complexity, also because the ISBSG dataset does not provide other thorough and consistent information about projects’ complexity.

Accordingly, we proceeded as follows:

1. For each project, we computed the proportion tf_{cplx} of high-complexity transactions over the total number of transactions.
2. We computed the $\frac{1}{3}$ and $\frac{2}{3}$ quantiles from the distribution of tf_{cplx} : let them be $tf_{1/3}$ and $tf_{2/3}$.
3. We selected the projects having $tf_{cplx} < tf_{1/3}$ as simple, those having $tf_{cplx} > tf_{2/3}$ as complex, and those with $tf_{1/3} \leq tf_{cplx} \leq tf_{2/3}$ as medium-complexity ones.

4.2.4. Scope Limitation

This paper deals with issues that stem from the actual usage of Function Points in practice. Specifically, some practitioners believe that IFPUG Function Points are better at estimating effort than simple metrics because the former incorporate the notion of complexity while the latter do not. To address this issue, we needed to consider the current practice and definitions, which are reflected in the ISBSG dataset.

It is well known that the definition of Function Points suffers from a few limitations [18]. For instance, the size of an EI transaction is constrained to be 3, 4, or 6; an EI that moves 10 DET through the boundary of the application has size 6, regardless of if it references 3 or 30 file types. This is because the “complexity” of all function types is measured via an ordinal scale, which includes *low*, *medium*, and *high* values. As a consequence, Function Points are not a ratio metric, with all the limitations that this entails [19].

However, the research questions that we address deal with the current definition and usage of Function Points, which ignore (or live along with) the aforementioned problems. So, we stick with the notion of complexity that is adopted by IFPUG Function Points; if we used a different (and theoretically more correct) definition of complexity, we would not be able to answer the research questions, which concern the definition of Function Points “as-is”. As a consequence of this simplification, and as discussed in Section 5, this study shows that weighting transactions and data according to an ordinal measure of complexity does not bring any practical advantage since simplified measures achieve approximately the same effort estimation accuracy as IFPUG Function Points.

4.3. Results for New Development Projects

In this section, the results of the analysis of new development projects are described. Section 4.3.1 illustrates the results obtained when considering all the new development projects from the ISBSG dataset, while Section 4.3.2 concerns only those projects that are more effort-consuming.

In both sections, all combinations of software complexity and productivity are considered: complexity is either ignored (i.e., all projects are considered together) or used to split the dataset into low-, mid-, and high-complexity ones; productivity is obtained as the mean or the median of ISBSG projects.

4.3.1. Results Obtained from All New Development Projects

Table 1 provides descriptive statistics for the new development projects contained in the dataset.

Table 1. Descriptive statistics of new development projects.

Metric	Size					Productivity	
	Mean	St. Dev.	Median	Min	Max	Mean	Median
UFP	542	619	312	6	3968	0.1838	0.1109
SFP	546	613	320	9	4250	0.1932	0.1129
tSFP	370	458	202	5	3123	0.1190	0.0674

Figure 1 shows the distribution of all new development projects’ effort in PHs.

When using mean productivity in model (1), we obtained estimation errors whose distribution is described in Figure 2; the mean and median absolute estimation errors are also given in the “all” columns of Table 2.

The boxplots in Figure 2 and the data in Table 2 indicate that the three considered measures yield quite similar errors. This was confirmed by the sign test, whose results are summarized in Table 3.

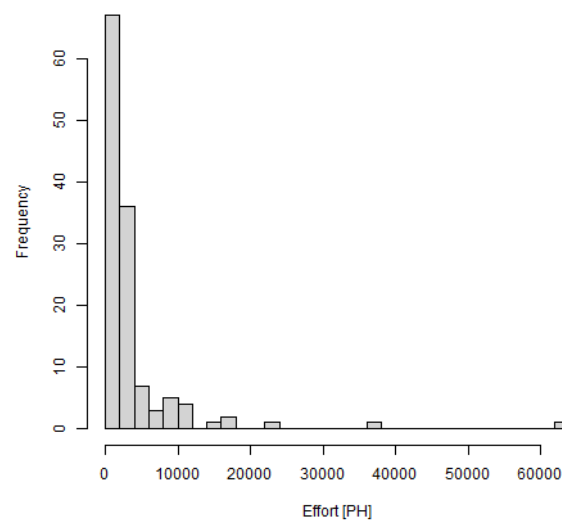


Figure 1. The distribution of all new development projects' effort in PHs.

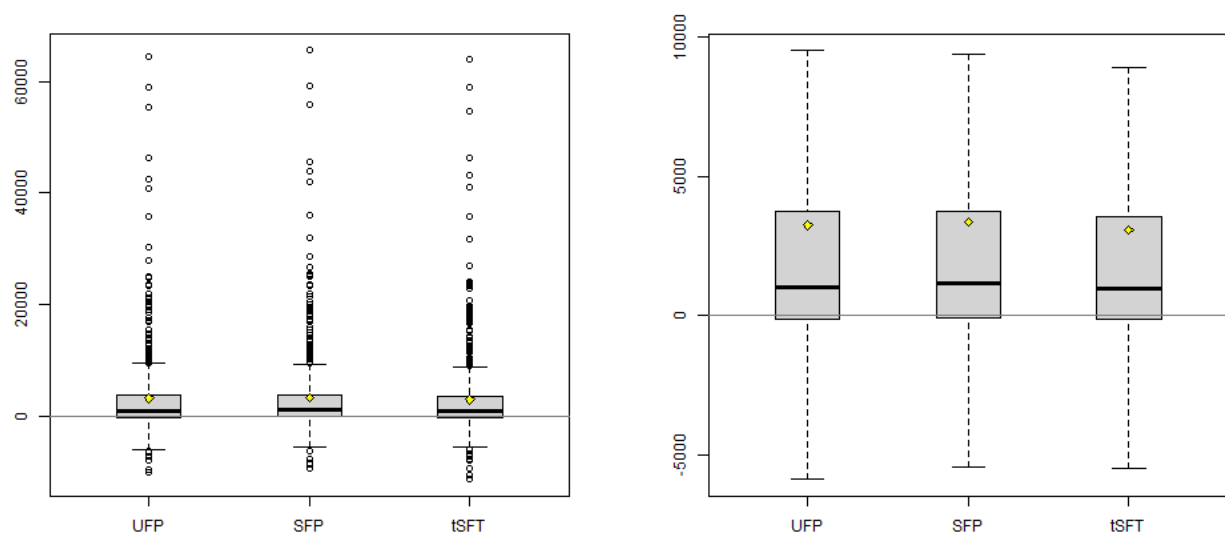


Figure 2. Boxplots of estimation errors with (left) and without outliers (right) for all new development projects when estimates are based on mean productivity.

Table 2. Mean and median absolute errors for new development projects when mean productivity is used.

	All		Low Complexity		Medium Complexity		High Complexity	
	MAR	MdAR	MAR	MdAR	MAR	MdAR	MAR	MdAR
UFP	4036	1636	2053	996	4695	2034	5377	2094
SFP	4112	1651	2045	1106	4724	1937	5585	2234
tSFP	4057	1676	2103	995	4618	1909	5467	2353

Table 3. Sign test results for all new development projects, mean productivity.

	UFP	SFP	tSFP
UFP	—	$\sim > (243 87 203)$	$= (241 45 247)$
SFP	$< (203 87 243)$	—	$= (240 41 252)$
tSFP	$= (247 45 241)$	$= (252 41 240)$	—

Each cell of Table 3 (and of the similar following tables) provides a symbol followed by three numbers in parentheses: the symbol indicates if the measure in the row was better, equivalent, or worse than the measure in the column, as described in Section 4.2.2; the numbers indicate how many times the measure in the row was better, equivalent, and worse, respectively, than the measure in the column. For instance, the cell in row UFP and column tSFP indicates that UFPs supported more accurate estimates for 241 projects and tSFPs supported more accurate estimates for 247 projects, while, for 45 projects, the accuracy difference was negligible.

When using median productivity in model (1), we obtained the estimation errors described in Figure 3; the mean and median absolute estimation errors are also given in the “all” columns of Table 4.

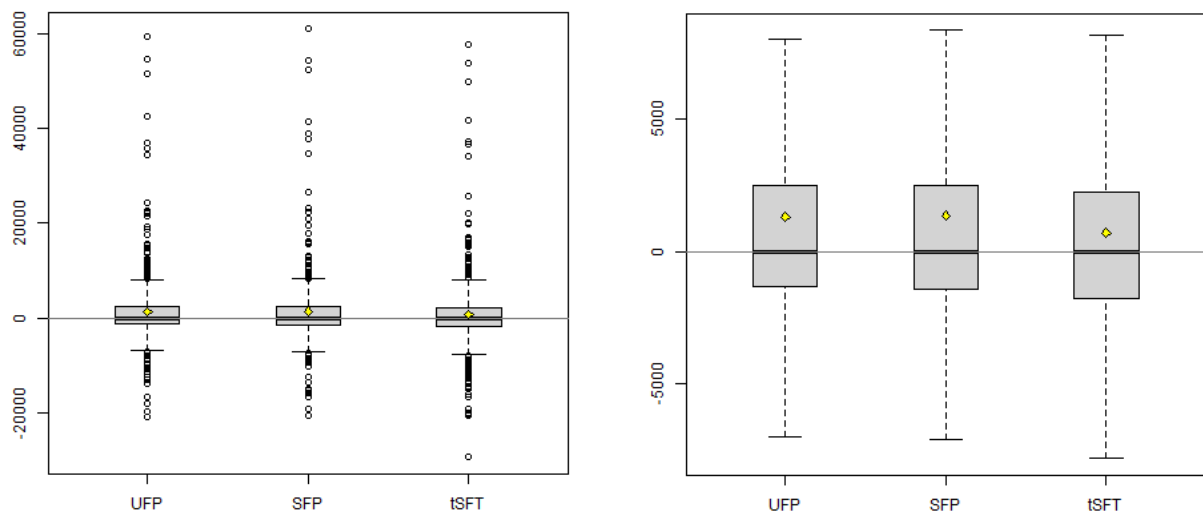


Figure 3. Boxplots of estimation errors with (left) and without outliers (right) for all new development projects when estimates are based on median productivity.

Table 4. Mean and median absolute errors for new development projects when median productivity is used.

	All		Low Complexity		Medium Complexity		High Complexity	
	MAR	MdAR	MAR	MdAR	MAR	MdAR	MAR	MdAR
UFP	3931	1774	2233	1016	4632	2197	4945	2174
SFP	3972	1799	2317	1209	4680	2387	4937	2008
tSFP	4329	1929	2604	1319	5346	2901	5057	2054

The results of the sign tests are summarized in Table 5.

Table 5. Sign test results for all new development projects, median productivity.

	UFP	SFP	tSFP
UFP	—	=(253 49 231)	~>(280 18 235)
SFP	=(231 49 253)	—	~>(280 20 233)
tSFP	<(235 18 280)	<(233 20 280)	—

We then proceeded to evaluate separately the high-, mid-, and low-complexity projects. As mentioned in Section 4.2, we computed the one-third and two-thirds percentiles from the distribution of the proportion tf_{cplx} of high-complexity transactions over the total number of transactions, obtaining $tf_{1/3} = 0.125$ and $tf_{2/3} = 0.36$. The new development projects of the ISBSG dataset are split by complexity as follows:

- A total of 179 low-complexity ($tf_{cplx} < tf_{1/3}$) projects.
- A total of 176 mid-complexity ($tf_{1/3} \leq tf_{cplx} \leq tf_{2/3}$) projects.
- A total of 178 high-complexity ($tf_{cplx} > tf_{2/3}$) projects.

The estimation errors obtained when using the mean productivity are shown in Figure 4.

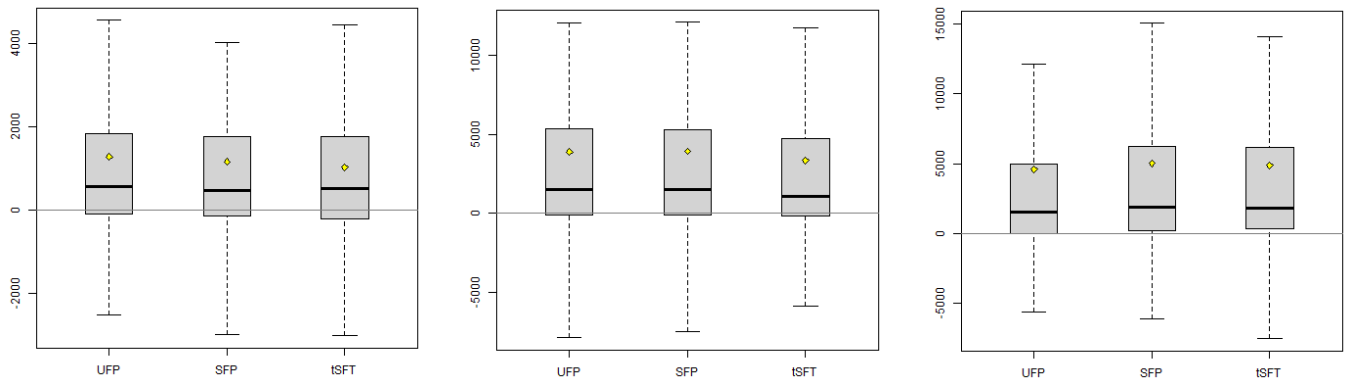


Figure 4. Boxplots of estimation errors for low (left), mid (center) and high (right) new development projects when estimates are based on mean productivity. Outliers omitted.

The results of the sign tests are summarized in Table 6.

Table 6. Sign test results for new development projects split per complexity, mean productivity.

	Low Complexity			Mid Complexity			High Complexity		
	UFP	SFP	tSFP	UFP	SFP	tSFP	UFP	SFP	tSFP
UFP	—	<(59 23 97)	=(81 7 91)	—	=(67 44 65)	<(68 17 91)	—	>(117 20 41)	~>(92 21 65)
SFP	~>(97 23 59)	—	=(88 7 84)	=(65 44 67)	—	<(68 18 90)	<(41 20 117)	—	=(84 16 78)
tSFP	=(91 7 81)	=(84 7 88)	—	~>(91 17 68)	~>(90 18 68)	—	<(65 21 92)	=(78 16 84)	—

The estimation errors obtained when using the median productivity are shown in Figure 5.

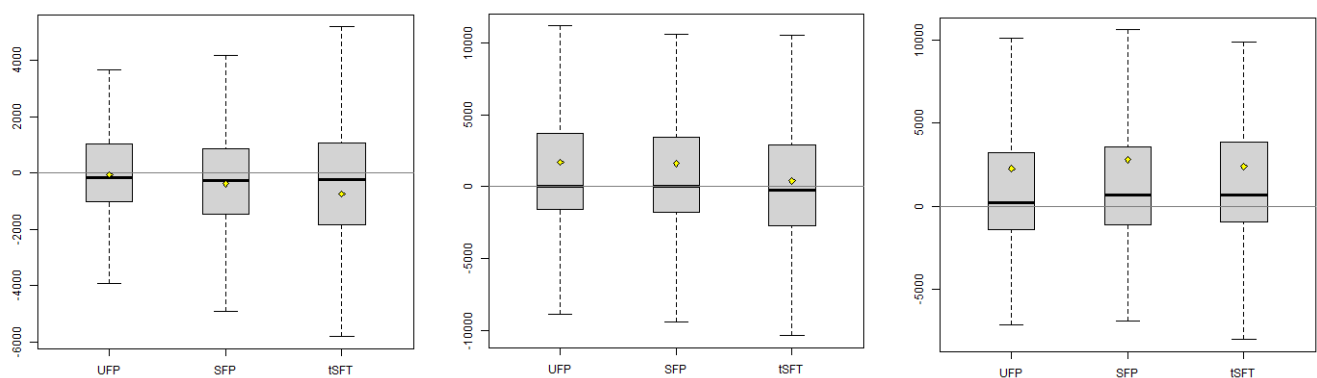


Figure 5. Boxplots of estimation errors for low (left), mid (center), and high (right) new development projects when estimates are based on median productivity. Outliers omitted.

The results of the sign tests are summarized in Table 7.

Table 7. Sign test results for new development projects split per complexity, median productivity.

	Low Complexity			Mid Complexity			High Complexity		
	UFP	SFP	tSFP	UFP	SFP	tSFP	UFP	SFP	tSFP
UFP	—	=(87 10 82)	>(105 11 73)	—	=(78 20 78)	=(85 9 82)	—	=(88 19 71)	=(90 8 80)
SFP	=(82 10 87)	—	>(102 5 72)	=(78 20 78)	—	=(89 7 80)	=(71 19 88)	—	=(89 8 81)
tSFP	<(73 11 105)	<(72 5 102)	—	=(82 9 85)	=(80 7 89)	—	=(80 8 90)	=(81 8 89)	—

4.3.2. Results Obtained from Selections of New Development Projects

As shown in Figure 1, the great majority of ISBSG new development projects required a relatively small effort. Specifically, 30% of the projects required no more than a PersonYear, while more than 50% required less than two PersonYears. We can thus conclude that the results reported in Section 4.3.1 are determined mainly by small (in terms of effort) projects. It is thus necessary to reconsider the research questions in the context of projects that require considerable development effort. To this end, we repeated the analysis described in Section 4.3.1, considering only projects that require considerable development effort. For the sake of space, in this section, we report only the results of the sign tests, while estimation error boxplots are omitted.

As a first step, we had to decide which projects should be involved in the analysis. We decided to retain the projects that required no less than two PersonYears, i.e., $2 \times 210 \times 8 = 3360$ PHs (assuming 210 working days per year and 8 working hours per day). In this way, we selected 247 projects. The descriptive statistics of this dataset are given in Table 8. In the rest of this paper, these projects are named conventionally “not too small”.

Table 8. Descriptive statistics of not-too-small new development projects.

Metric	Size					Productivity	
	Mean	St. Dev.	Median	Min	Max	Mean	Median
UFP	703	654	445	51	3968	0.1973	0.1143
SFP	706	648	465	51	4250	0.2049	0.1140
tSFP	482	490	299	23	3123	0.1280	0.0701

Mean and median absolute errors for not-so-small new development projects, when mean productivity is used, are given in columns “all” of Table 9, while the sign tests applied to absolute residuals yielded the results summarized in Table 10.

Table 9. Mean and median absolute errors for not-so-small new development projects when mean productivity is used.

	All		Low Complexity		Medium Complexity		High Complexity	
	MAR	MdAR	MAR	MdAR	MAR	MdAR	MAR	MdAR
UFP	5330	2531	3255	1845	5815	2950	6922	3196
SFP	5424	2629	3241	1775	5849	3232	7183	3407
tSFP	5344	2701	3252	1765	5717	3313	7065	3178

Table 10. Sign test results for not-too-small new development projects, mean productivity.

	UFP	SFP	tSFP
UFP	—	~>(178 75 136)	=(173 38 178)
SFP	<(136 75 178)	—	=(165 32 192)
tSFP	=(178 38 173)	=(192 32 165)	—

When using mean productivity in model (1) and applying the model to the dataset split by complexity, we obtained the mean and median absolute estimation errors described in the rightmost columns of Table 9; the sign tests applied to absolute residuals yielded the results summarized in Table 11.

Table 11. Sign test results for not-too-small new development projects split per complexity, mean productivity.

	Low Complexity			Mid Complexity			High Complexity		
	UFP	SFP	tSFP	UFP	SFP	tSFP	UFP	SFP	tSFP
UFP	—	<(42 16 72)	=(56 7 67)	—	=(54 36 39)	=(50 14 65)	—	>(82 23 25)	~>(67 17 46)
SFP	~>(72 16 42)	—	=(59 6 65)	=(39 36 54)	—	<(47 14 68)	<(25 23 82)	—	=(59 12 59)
tSFP	=(67 7 56)	=(65 6 59)	—	=(65 14 50)	~>(68 14 47)	—	<(46 17 67)	=(59 12 59)	—

Mean and median absolute errors for not-so-small new development projects, when median productivity is used, are given in columns “all” of Table 12, while the sign tests applied to absolute residuals yielded the results summarized in Table 13.

Table 12. Mean and median absolute errors for not-so-small new development projects when median productivity is used.

	All		Low Complexity		Medium Complexity		High Complexity	
	MAR	MdAR	MAR	MdAR	MAR	MdAR	MAR	MdAR
UFP	5086	2973	3293	2213	5782	3347	6189	3180
SFP	5172	2884	3469	2396	5843	3397	6211	3302
tSFP	5564	3036	3938	2431	6391	3690	6370	3890

Table 13. Sign test results for not-too-small new development projects, median productivity.

	UFP	SFP	tSFP
UFP	—	~>(200 43 146)	=(198 17 174)
SFP	<(146 43 200)	—	=(201 14 174)
tSFP	=(174 17 198)	=(174 14 201)	—

When using median productivity in model (1) and applying the model to the dataset split by complexity, we obtained the mean and median absolute estimation errors described in the rightmost columns of Table 12; the sign tests applied to absolute residuals yielded the results summarized in Table 14.

Table 14. Sign test results for not-too-small new development projects split per complexity, median productivity.

	Low Complexity			Mid Complexity			High Complexity		
	UFP	SFP	tSFP	UFP	SFP	tSFP	UFP	SFP	tSFP
UFP	—	~>(72 9 49)	=(73 2 55)	—	=(61 19 49)	=(59 6 64)	—	~>(67 15 48)	=(66 9 55)
SFP	<(49 9 72)	—	=(72 2 56)	=(49 19 61)	—	=(62 5 62)	<(48 15 67)	—	=(67 7 56)
tSFP	=(55 2 73)	=(56 2 72)	—	=(64 6 59)	=(62 5 62)	—	=(55 9 66)	=(56 7 67)	—

4.4. Results for Extension Projects

In this section, the results of the analysis of extension projects are described. Section 4.4.1 illustrates the results obtained when considering all the extension projects from the ISBSG dataset, while Section 4.4.2 concerns only those projects that are more effort-consuming.

In both sections, all combinations of software complexity and productivity are considered: complexity is either ignored (i.e., all projects are considered together) or used to split the dataset into low-, mid-, and high-complexity ones; productivity is obtained as the mean or the median of ISBSG projects.

4.4.1. Results Obtained from All Extension Projects

Table 15 provides descriptive statistics for the extension projects contained in the dataset.

Table 15. Descriptive statistics of extension projects.

Metric	Size					Productivity	
	Mean	St.Dev.	Median	Min	Max	Mean	Median
UFP	214	221	145	9	1239	0.1297	0.0810
SFP	216	227	148	9	1405	0.1296	0.0863
tSFP	150	169	97	9	1118	0.0867	0.0590

Figure 6 shows the distribution of all extension projects' effort in PHs.

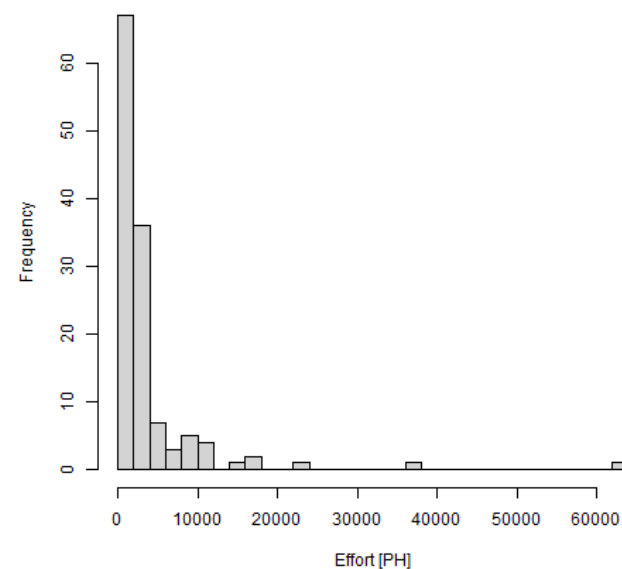


Figure 6. The distribution of all extension projects' effort in PHs.

When using mean productivity in model (1), we obtained estimation errors whose distribution is described in Figure 7; the mean and median absolute estimation errors are also given in the “all” columns of Table 16.

Table 16. Mean and median absolute errors for extension projects when mean productivity is used.

	All		Low Complexity		Medium Complexity		High Complexity	
	MAR	MdAR	MAR	MdAR	MAR	MdAR	MAR	MdAR
UFP	2597	798	1495	740	2634	1119	3663	579
SFP	2565	717	1406	640	2566	1076	3723	572
tSFP	2568	764	1579	598	2582	1031	3544	567

The boxplots in Figure 7 and the data in Table 16 indicate that the three considered measures yield quite similar errors. This was confirmed by the sign test, whose results are summarized in Table 17.

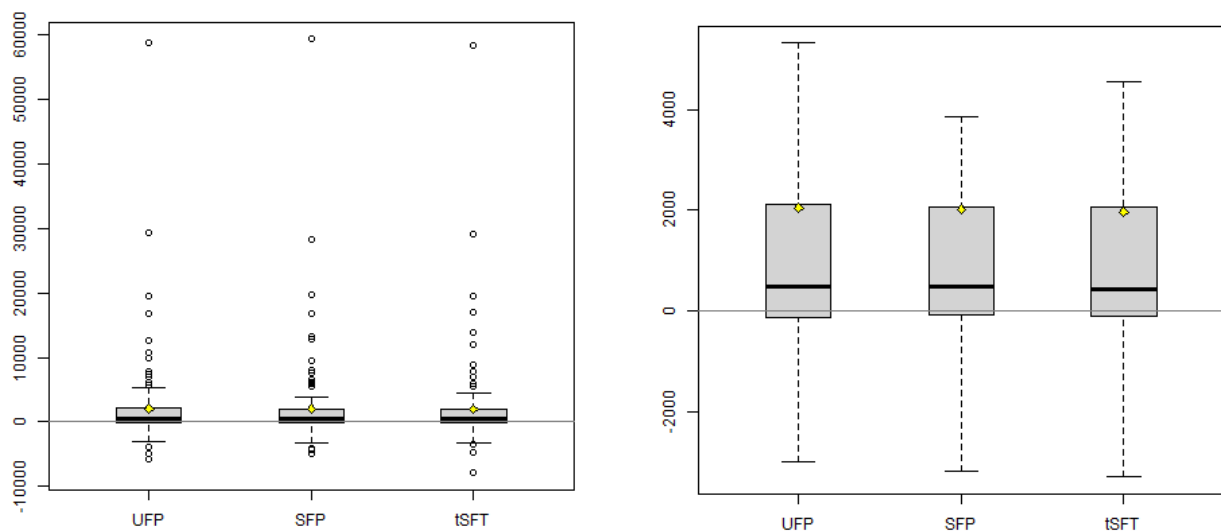


Figure 7. Boxplots of estimation errors with (left) and without outliers (right) for all new development projects when estimates are based on mean productivity.

Table 17. Sign test results for all extension projects, mean productivity.

	UFP	SFP	tsSFT
UFP	—	<(44 17 67)	=(50 12 66)
SFP	~>(67 17 44)	—	=(56 15 57)
tsSFT	=(66 12 50)	=(57 15 56)	—

When using median productivity in model (1), we obtained the estimation errors described in Figure 8; the mean and median absolute estimation errors are also given in the “all” columns of Table 18.

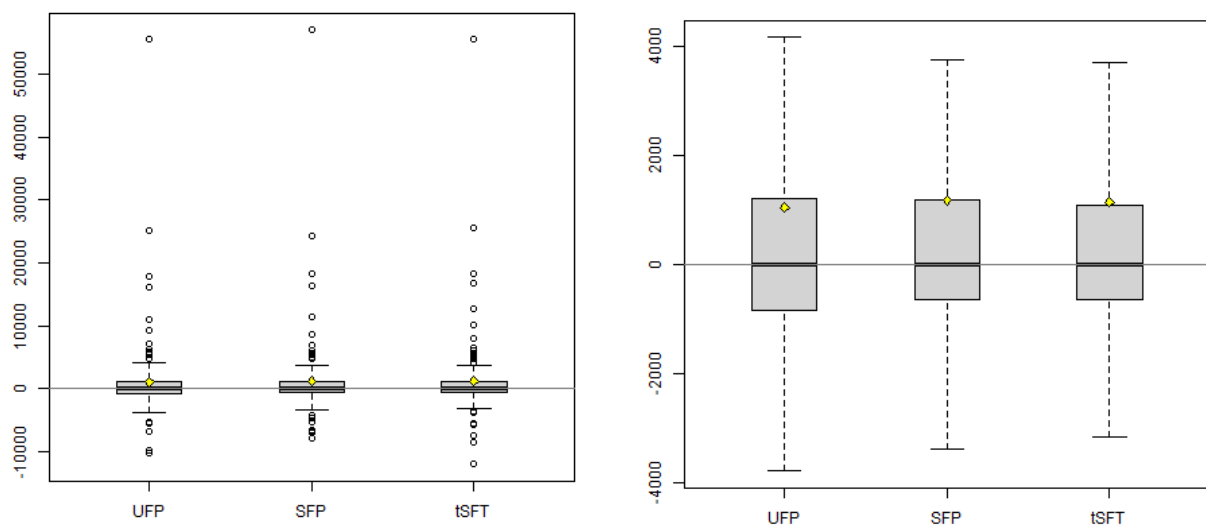


Figure 8. Boxplots of estimation errors with (left) and without outliers (right) for all extension projects when estimates are based on median productivity.

Table 18. Mean and median absolute errors for extension projects when median productivity is used.

	All MAR	All MdAR	Low Complexity MAR	Low Complexity MdAR	Medium Complexity MAR	Medium Complexity MdAR	High Complexity MAR	High Complexity MdAR
UFP	2593	900	1566	730	2581	964	3632	998
SFP	2566	881	1601	549	2443	936	3652	992
tSFP	2582	853	1904	698	2463	1077	3375	632

The results of the sign tests are summarized in Table 19.

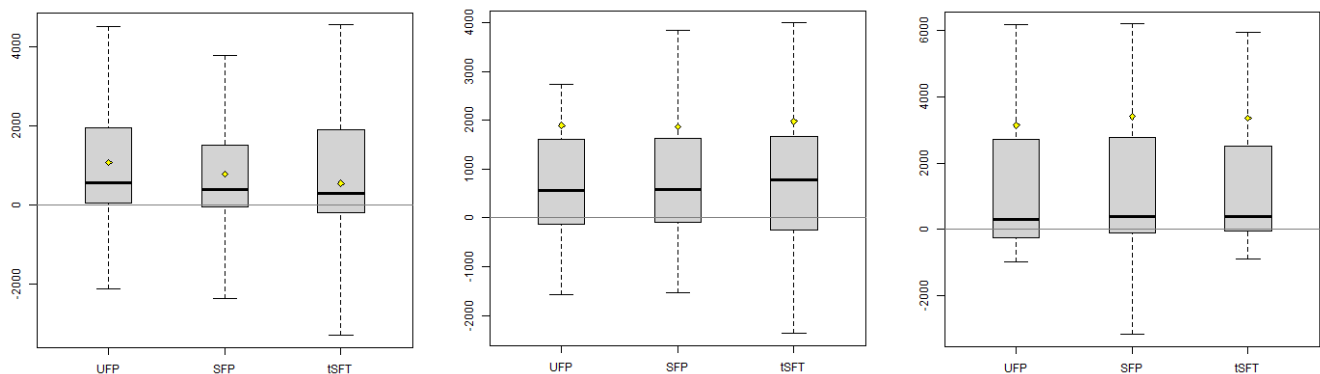
Table 19. Sign test results for all extension projects, median productivity.

	UFP	SFP	tSFP
UFP	—	=(57 7 64)	=(61 5 62)
SFP	=(64 7 57)	—	=(59 5 64)
tSFP	=(62 5 61)	=(64 5 59)	—

We then proceeded to evaluate separately the high-, mid-, and low-complexity projects. As mentioned in Section 4.2, we computed the one-third and two-thirds percentiles from the distribution of the proportion tf_{cplx} of high-complexity transactions over the total number of transactions, obtaining $tf_{1/3} = 0.174$ and $tf_{2/3} = 0.38$. The extension projects of the ISBSG dataset are split by complexity as follows:

- A total of 43 low-complexity ($tf_{cplx} < tf_{1/3}$) projects.
- A total of 42 mid-complexity ($tf_{1/3} \leq tf_{cplx} \leq tf_{2/3}$) projects.
- A total of 43 high-complexity ($tf_{cplx} > tf_{2/3}$) projects.

The estimation errors obtained when using the mean productivity are shown in Figure 9; the mean and median absolute estimation errors are also given in the rightmost columns of Table 16.

**Figure 9.** Boxplots of estimation errors for low (left), mid (center), and high (right) extension projects when estimates are based on mean productivity. Outliers omitted.

The results of the sign tests are summarized in Table 20.

Table 20. Sign test results for extension projects split per complexity, mean productivity.

	Low Complexity			Mid Complexity			High Complexity		
	UFP	SFP	tSFP	UFP	SFP	tSFP	UFP	SFP	tSFP
UFP	—	<(11 2 30)	=(17 1 25)	—	=(14 10 18)	=(21 5 16)	—	=(19 5 19)	<(12 6 25)
SFP	>(30 2 11)	—	=(21 4 18)	=(18 10 14)	—	~>(25 5 12)	=(19 5 19)	—	<(10 6 27)
tSFP	=(25 1 17)	=(18 4 21)	—	=(16 5 21)	<(12 5 25)	—	~>(25 6 12)	~>(27 6 10)	—

The estimation errors obtained when using the median productivity are shown in Figure 10; the mean and median absolute estimation errors are also given in the rightmost columns of Table 18.

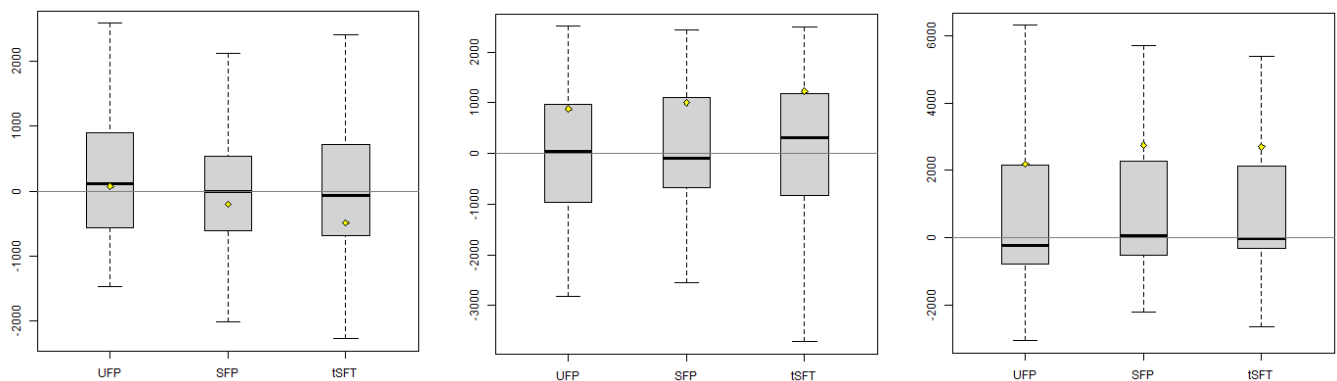


Figure 10. Boxplots of estimation errors for low (left), mid (center), and high (right) extension projects when estimates are based on median productivity. Outliers omitted.

The results of the sign tests are summarized in Table 21.

Table 21. Sign test results for extension projects split per complexity, median productivity.

	Low Complexity			Mid Complexity			High Complexity		
	UFP	SFP	tSFP	UFP	SFP	tSFP	UFP	SFP	tSFP
UFP	—	=(20 4 19)	=(22 2 19)	—	=(17 2 23)	=(22 2 18)	—	=(20 1 22)	=(17 1 25)
SFP	=(19 4 20)	—	=(22 1 20)	=(23 2 17)	—	=(25 2 15)	=(22 1 20)	—	<(12 2 29)
tSFP	=(19 2 22)	=(20 1 22)	—	=(18 2 22)	=(15 2 25)	—	=(25 1 17)	>(29 2 12)	—

4.4.2. Results Obtained from Selections of Extension Projects

As shown in Figure 6, the great majority of ISBSG extension projects required a relatively small effort. Specifically, 50% required less than one PersonYear. We can thus conclude that the results reported in Section 4.4.1 are determined mainly by small (in terms of effort) projects. It is thus necessary to reconsider the research questions in the context of projects that require considerable development effort. To this end, we repeated the analysis described in Section 4.4.1, considering only projects that require considerable development effort. For the sake of space, in this section, we report only the results of the sign tests, while estimation error boxplots are omitted.

As a first step, we had to decide which projects should be involved in the analysis. We decided to retain the projects that required no less than one PersonYears, i.e., $210 \times 8 = 1680$ PHs (assuming 210 working days per year and 8 working hours per day). In this way, we selected 64 projects. The descriptive statistics of this dataset are given in Table 22. In the rest of this paper, these projects are named conventionally “not too small”.

Table 22. Descriptive statistics of not-too-small extension projects.

Metric	Size					Productivity	
	Mean	St. Dev.	Median	Min	Max	Mean	Median
UFP	277	234	201	10	1239	0.1266	0.0776
SFP	280	242	199	9	1405	0.1264	0.0773
tSFP	194	182	133	9	1118	0.0853	0.0513

Mean and median absolute errors for not-so-small enhancement projects, when mean productivity is used, are given in columns “all” of Table 23, while the sign tests applied to absolute residuals yielded the results summarized in Table 24.

Table 23. Mean and median absolute errors for not-so-small extension projects when mean productivity is used.

	All		Low Complexity		Medium Complexity		High Complexity	
	MAR	MdAR	MAR	MdAR	MAR	MdAR	MAR	MdAR
UFP	3503	1559	2120	1672	2903	1298	5505	2586
SFP	3474	1538	2018	1635	2867	1170	5558	2610
tSFP	3505	1395	2324	1882	2912	1165	5298	2111

Table 24. Sign test results for not-too-small extension projects, mean productivity.

	UFP	SFP	tSFP
UFP	—	=(34 13 44)	=(40 12 39)
SFP	=(44 13 34)	—	=(42 9 40)
tSFP	=(39 12 40)	=(40 9 42)	—

When using mean productivity in model (1) and applying the model to the dataset split by complexity, we obtained the mean and median absolute estimation errors described in the rightmost columns of Table 23; the sign tests applied to absolute residuals yielded the results summarized in Table 25.

Table 25. Sign test results for not-too-small extension projects split per complexity, mean productivity.

	Low Complexity			Mid Complexity			High Complexity		
	UFP	SFP	tSFP	UFP	SFP	tSFP	UFP	SFP	tSFP
UFP	—	=(10 2 18)	=(14 1 15)	—	=(10 6 15)	=(17 5 9)	—	=(14 5 11)	=(9 6 15)
SFP	=(18 2 10)	—	=(17 2 11)	=(15 6 10)	—	~>(20 2 9)	=(11 5 14)	—	<(5 5 20)
tSFP	=(15 1 14)	=(11 2 17)	—	=(9 5 17)	<(9 2 20)	—	=(15 6 9)	>(20 5 5)	—

Mean and median absolute errors for not-so-small enhancement projects, when median productivity is used, are given in columns “all” of Table 26, while the sign tests applied to absolute residuals yielded the results summarized in Table 27.

Table 26. Mean and median absolute errors for not-so-small extension projects when median productivity is used.

	All		Low Complexity		Medium Complexity		High Complexity	
	MAR	MdAR	MAR	MdAR	MAR	MdAR	MAR	MdAR
UFP	3528	1746	2245	1667	2883	1315	5478	2129
SFP	3550	1629	2415	1551	2817	1214	5442	2147
tSFP	3633	1756	3067	1952	2812	1410	5047	2234

Table 27. Sign test results for not-too-small extension projects, median productivity.

	UFP	SFP	tSFP
UFP	—	=(40 7 44)	=(39 6 46)
SFP	=(44 7 40)	—	=(44 5 42)
tSFP	=(46 6 39)	=(42 5 44)	—

When using median productivity in model (1) and applying the model to the dataset split by complexity, we obtained the mean and median absolute estimation errors described in the rightmost columns of Table 26; the sign tests applied to absolute residuals yielded the results summarized in Table 28.

Table 28. Sign test results for not-too-small extension projects split per complexity, median productivity.

	Low Complexity			Mid Complexity			High Complexity		
	UFP	SFP	tSFP	UFP	SFP	tSFP	UFP	SFP	tSFP
UFP	—	=(15 2 13)	=(16 0 14)	—	=(11 2 18)	=(13 4 14)	—	=(14 3 13)	=(10 2 18)
SFP	=(13 2 15)	—	=(18 2 10)	=(18 2 11)	—	=(17 1 13)	=(13 3 14)	—	<(9 2 19)
tSFP	=(14 0 16)	=(10 2 18)	—	=(14 4 13)	=(13 1 17)	—	=(18 2 10)	~>(19 2 9)	—

4.5. Results for Enhancement Projects

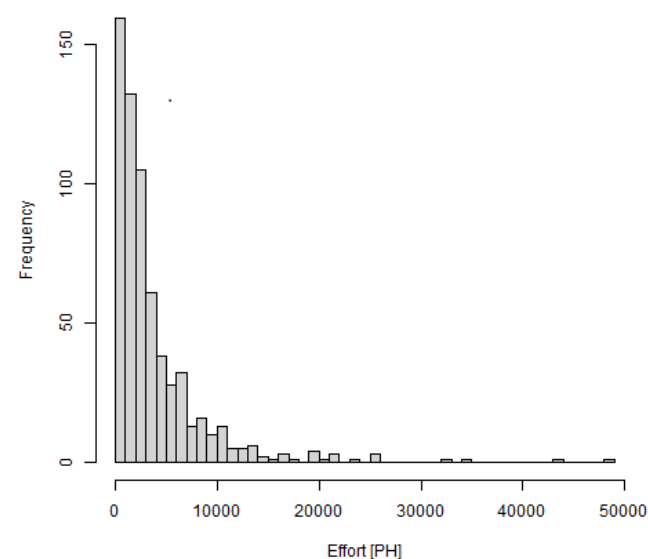
In this section, the results of the analysis of enhancement projects are described. Section 4.5.1 illustrates the results obtained when considering all the enhancement projects from the ISBSG dataset, while Section 4.5.2 concerns only those projects that are more effort-consuming.

Table 29 provides descriptive statistics for the enhancement projects contained in the dataset.

Table 29. Descriptive statistics of enhancement projects.

Metric	Size					Productivity	
	Mean	St. Dev.	Median	Min	Max	Mean	Median
UFP	322	497	185	4	7134	0.1539	0.0787
SFP	313	489	175	5	7157	0.1521	0.0795
tSFP	236	355	131	5	3993	0.1106	0.0581

Figure 11 shows the distribution of all enhancement projects' effort in PHs.

**Figure 11.** The distribution of all enhancement projects' effort in PHs.

4.5.1. Results Obtained from All Enhancement Projects

When using mean productivity in model (1), we obtained estimation errors whose distribution is described in Figure 12; the mean and median absolute estimation errors are also given in the “all” columns of Table 30.

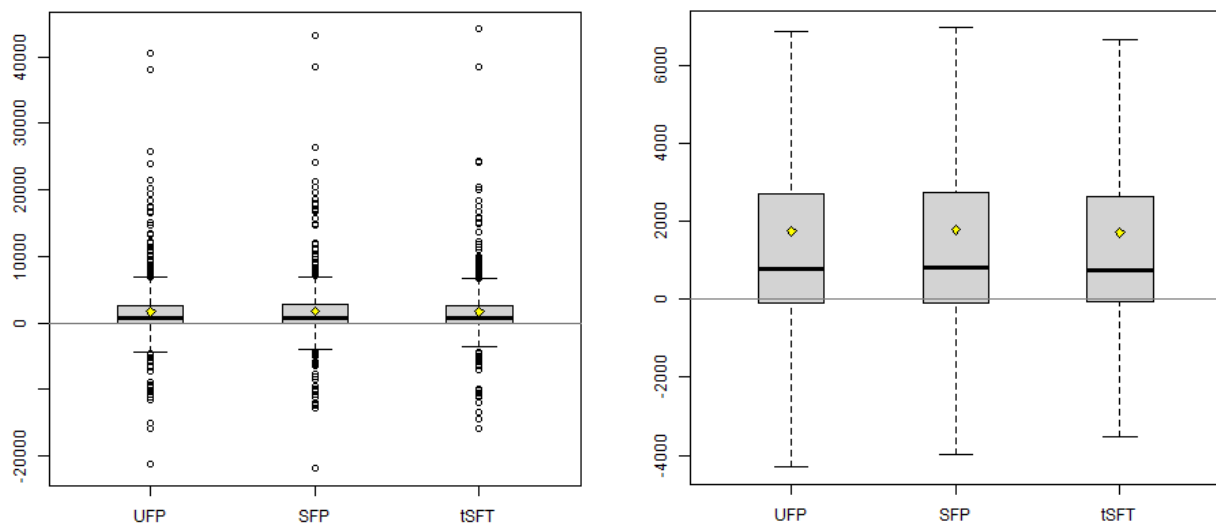


Figure 12. Boxplots of estimation errors with (left) and without outliers (right) for all enhancement projects when estimates are based on mean productivity.

Table 30. Mean and median absolute errors for enhancement projects when mean productivity is used.

	All		Low Complexity		Medium Complexity		High Complexity	
	MAR	MdAR	MAR	MdAR	MAR	MdAR	MAR	MdAR
UFP	2790	1399	2291	1080	2915	1592	3162	1406
SFP	2819	1342	2299	1103	2928	1668	3228	1312
tSFP	2725	1351	2207	899	2749	1444	3219	1352

The results of the sign test applied to absolute errors are summarized in Table 31.

Table 31. Sign test results for all enhancement projects, mean productivity.

	UFP	SFP	tSFP
UFP	—	$\approx (301 82 263)$	$< (256 72 318)$
SFP	$\approx (263 82 301)$	—	$< (234 56 356)$
tSFP	$\sim > (318 72 256)$	$> (356 56 234)$	—

When using median productivity in model (1), we obtained the estimation errors described in Figure 13; the mean and median absolute estimation errors are also given in the “all” columns of Table 32.

Table 32. Mean and median absolute errors for enhancement projects when median productivity is used.

	All		Low Complexity		Medium Complexity		High Complexity	
	MAR	MdAR	MAR	MdAR	MAR	MdAR	MAR	MdAR
UFP	3120	1422	2630	1130	3382	1694	3346	1455
SFP	3117	1413	2784	1112	3358	1699	3209	1522
tSFP	3077	1509	2727	1130	3160	1736	3343	1503

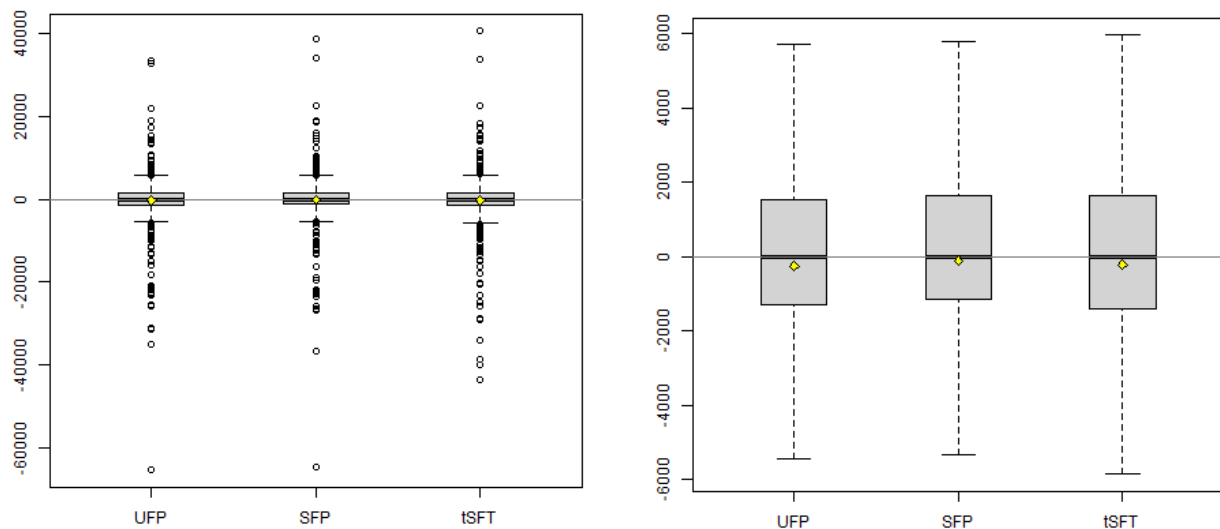


Figure 13. Boxplots of estimation errors with (left) and without outliers (right) for all enhancement projects when estimates are based on median productivity.

The results of the sign tests applied to absolute estimation errors are summarized in Table 33.

Table 33. Sign test results for all new development projects, median productivity.

	UFP	SFP	tSFP
UFP	—	=(321 40 285)	>(350 27 269)
SFP	=(285 40 321)	—	=(322 30 294)
tSFP	<(269 27 350)	=(294 30 322)	—

We then proceeded to evaluate separately the high-, mid-, and low-complexity projects. As mentioned in Section 4.2, we computed the one-third and two-thirds percentiles from the distribution of the proportion tf_{cplx} of high-complexity transactions over the total number of transactions, obtaining $tf_{1/3} = 0$ and $tf_{2/3} = 0.23$. The enhancement projects of the ISBSG dataset are split by complexity as follows:

- A total of 215 low-complexity ($tf_{cplx} < tf_{1/3}$) projects.
- A total of 216 mid-complexity ($tf_{1/3} \leq tf_{cplx} \leq tf_{2/3}$) projects.
- A total of 215 high-complexity ($tf_{cplx} > tf_{2/3}$) projects.

The estimation errors obtained when using the mean productivity are shown in Figure 14; the mean and median absolute estimation errors described in the rightmost columns of Table 30; the sign tests applied to absolute residuals yielded the results summarized in Table 34.

Table 34. Sign test results for enhancement projects split per complexity, mean productivity.

	Low Complexity			Mid Complexity			High Complexity		
	UFP	SFP	tSFP	UFP	SFP	tSFP	UFP	SFP	tSFP
UFP	—	<(67 21 127)	<(84 7 124)	—	=(86 49 81)	<(77 19 120)	—	>(148 12 55)	=(95 46 74)
SFP	>(127 21 67)	—	=(89 15 111)	=(81 49 86)	—	<(74 21 121)	<(55 12 148)	—	<(71 20 124)
tSFP	>(124 7 84)	=(111 15 89)	—	~>(120 19 77)	>(121 21 74)	—	=(74 46 95)	>(124 20 71)	—

The estimation errors obtained when using the median productivity are shown in Figure 15; the mean and median absolute estimation errors described in the rightmost columns of Table 32; the sign tests applied to absolute residuals yielded the results summarized in Table 35.

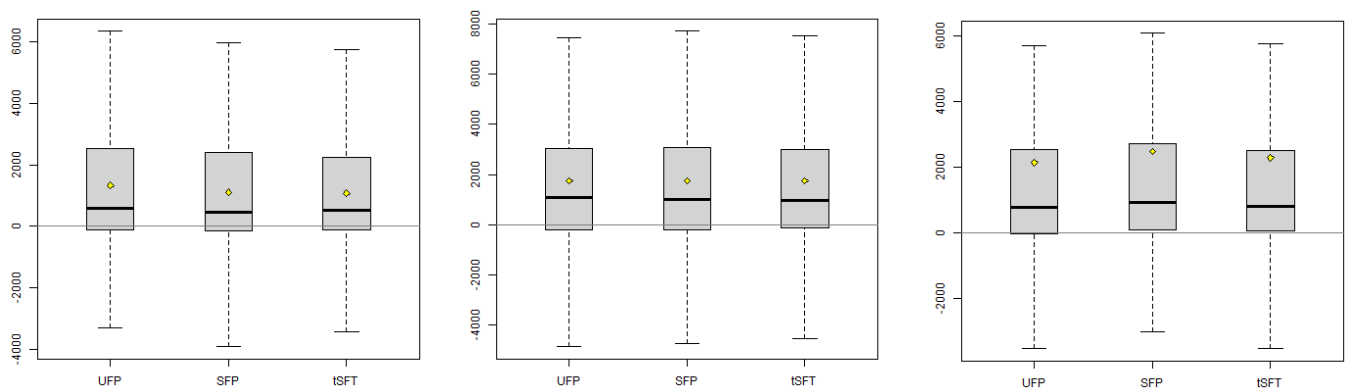


Figure 14. Boxplots of estimation errors for low (left), mid (center), and high (right) enhancement projects, when estimates are based on mean productivity. Outliers omitted.

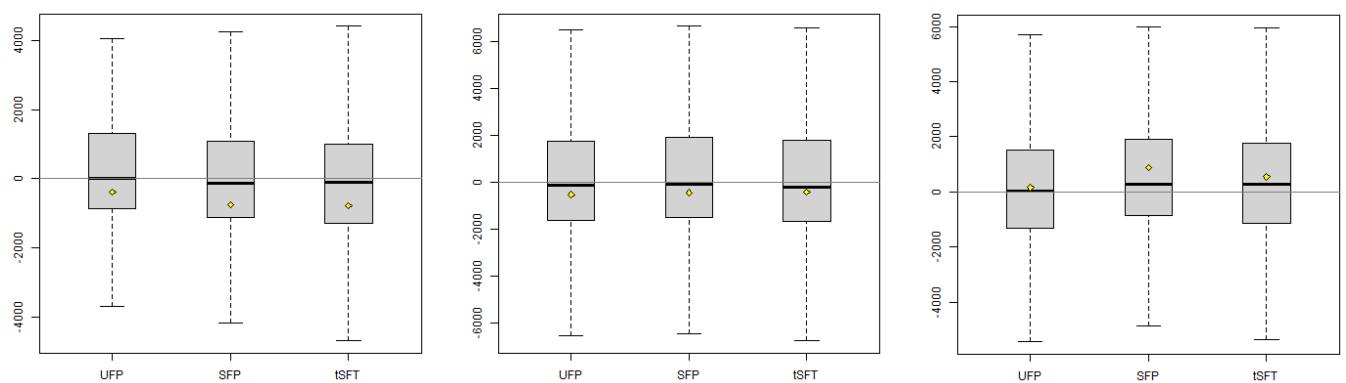


Figure 15. Boxplots of estimation errors for low (left), mid (center), and high (right) enhancement projects when estimates are based on median productivity. Outliers omitted.

Table 35. Sign test results for enhancement projects split per complexity, median productivity.

	Low Complexity			Mid Complexity			High Complexity		
	UFP	SFP	tSFP	UFP	SFP	tSFP	UFP	SFP	tSFP
UFP	—	~>(116 12 87)	~>(119 4 92)	—	=(96 26 94)	=(115 8 93)	—	=(109 2 104)	~>(116 15 84)
SFP	<(87 12 116)	—	=(105 8 102)	=(94 26 96)	—	=(111 12 93)	=(104 2 109)	—	=(106 10 99)
tSFP	<(92 4 119)	=(102 8 105)	—	=(93 8 115)	=(93 12 111)	—	<(84 15 116)	=(99 10 106)	—

4.5.2. Results Obtained from Selections of Enhancement Projects

As shown in Figure 11, the great majority of ISBSG enhancement projects required a relatively small effort. Specifically, over 30% required less than one PersonYear, while over 64% required no more than two PersonYears. We can thus conclude that the results reported in Section 4.5.1 are determined mainly by small (in terms of effort) projects. It is thus necessary to reconsider the research questions in the context of projects that require considerable development effort. To this end, we repeated the analysis described in Section 4.5.1, considering only enhancement projects that require considerable development effort. For the sake of space, in this section, we report only the results of the sign tests, while estimation error boxplots are omitted.

As a first step, we had to decide which projects should be involved in the analysis. We decided to retain the projects that required no less than one PersonYear, i.e., $210 \times 8 = 1680$ PHs (assuming 210 working days per year and 8 working hours per day). In this way, we selected 254 projects. The descriptive statistics of this dataset are given in Table 36. In the rest of this paper, these projects are named conventionally “not too small”.

Table 36. Descriptive statistics of not-too-small enhancement projects.

Metric	Size					Productivity	
	Mean	St. Dev.	Median	Min	Max	Mean	Median
UFP	390	536	236	7	7134	0.1535	0.0767
SFP	379	528	237	5	7157	0.1510	0.0764
tSFP	285	382	184	5	3993	0.1115	0.0569

Mean and median absolute errors for not-so-small enhancement projects when mean productivity is used are given in columns “all” of Table 37, while the sign tests applied to absolute residuals yielded the results summarized in Table 38.

Table 37. Mean and median absolute errors for not-so-small enhancement projects when mean productivity is used.

	All		Low Complexity		Medium Complexity		High Complexity	
	MAR	MdAR	MAR	MdAR	MAR	MdAR	MAR	MdAR
UFP	3402	1992	3120	2110	3193	1860	3891	1999
SFP	3437	1988	3142	2024	3204	1886	3963	1985
tSFP	3323	1974	2979	2063	3024	1728	3963	2004

Table 38. Sign test results for not-too-small enhancement projects, mean productivity.

	UFP	SFP	tSFP
UFP	—	<(54 21 98)	<(68 9 96)
SFP	>(98 21 54)	—	=(69 16 88)
tSFP	~>(96 9 68)	=(88 16 69)	—

When using mean productivity in model (1) and applying the model to the dataset split by complexity, we obtained the mean and median absolute estimation errors described in the rightmost columns of Table 37; the sign tests applied to absolute residuals yielded the results summarized in Table 39.

Table 39. Sign test results for not-too-small enhancement projects split per complexity, mean productivity.

	Low Complexity			Mid Complexity			High Complexity		
	UFP	SFP	tSFP	UFP	SFP	tSFP	UFP	SFP	tSFP
UFP	—	<(16 1 35)	=(21 1 30)	—	=(17 5 13)	=(15 0 20)	—	>(34 1 9)	=(18 4 22)
SFP	>(35 1 16)	—	=(25 3 24)	=(13 5 17)	—	=(13 1 21)	<(9 1 34)	—	<(12 1 31)
tSFP	=(30 1 21)	=(24 3 25)	—	=(20 0 15)	=(21 1 13)	—	=(22 4 18)	>(31 1 12)	—

Mean and median absolute errors for not-so-small enhancement projects when median productivity is used are given in columns “all” of Table 40, while the sign tests applied to absolute residuals yielded the results summarized in Table 41.

Table 40. Mean and median absolute errors for not-so-small enhancement projects when median productivity is used.

	All		Low Complexity		Medium Complexity		High Complexity	
	MAR	MdAR	MAR	MdAR	MAR	MdAR	MAR	MdAR
UFP	3873	2010	3633	2034	3815	2124	4170	1826
SFP	3885	2054	3875	2103	3784	1945	3996	2061
tSFP	3801	2033	3678	2108	3579	2067	4146	1947

Table 41. Sign test results for not-too-small enhancement projects, median productivity.

	UFP	SFP	tSFP
UFP	—	=(86 10 77)	=(94 6 73)
SFP	=(77 10 86)	—	=(87 11 75)
tSFP	=(73 6 94)	=(75 11 87)	—

When using median productivity in model (1) and applying the model to the dataset split by complexity, we obtained the mean and median absolute estimation errors described in the rightmost columns of Table 40; the sign tests applied to absolute residuals yielded the results summarized in Table 42.

Table 42. Sign test results for not-too-small enhancement projects split per complexity, median productivity.

	UFP	Low Complexity SFP	tSFP	UFP	Mid Complexity SFP	tSFP	UFP	High Complexity SFP	tSFP
UFP	—	=(30 1 21)	>(34 0 18)	—	=(14 3 18)	=(14 0 21)	—	=(22 0 22)	=(20 3 21)
SFP	=(21 1 30)	—	=(29 0 23)	=(18 3 14)	—	=(14 1 20)	=(22 0 22)	—	=(24 1 19)
tSFP	<(18 0 34)	=(23 0 29)	—	=(21 0 14)	=(20 1 14)	—	=(21 3 20)	=(19 1 24)	—

5. Discussion

In this section, we answer the research questions enunciated in Section 3. Having considered two simple functional size measures (SFPs and tSFPs), we answer each question separately for UFPs vs. SFPs and UFPs vs. tSFPs.

5.1. Answer to RQ1

Research question RQ1 asks if simple functional measures (namely, SFPs and tSFPs) provide effort estimates that are as accurate as those provided by IFPUG UFPs when project complexity is not taken into account.

Table 43 summarizes the results illustrated in Section 4.3.1 that are relevant for RQ1.

Table 43. Summary of results when complexity is not considered.

Project Type	Subset	Productivity	UFP vs. SFP	UFP vs. tSFP
New dev.	All	mean	~>	=
New dev.	All	median	=	~>
New dev.	"Not too small"	mean	~>	=
New dev.	"Not too small"	median	~>	=
Extensions	All	mean	<	=
Extensions	All	median	=	=
Extensions	"Not too small"	mean	=	=
Extensions	"Not too small"	median	=	=
Enhancements	All	mean	=	<
Enhancements	All	median	=	>
Enhancements	"Not too small"	mean	<	<
Enhancements	"Not too small"	median	=	=

It is easy to see that, in most cases (15 out of 24), a simplified measure supports effort estimation at the same accuracy level as IFPUG UFPs. Noticeably, this is true also if SFPs and tSFPs are considered separately: SFPs are equivalent to UFPs in 7 cases out of 12, better in 2 cases, and worse in 3 cases; tSFPs are equivalent to UFPs in 8 cases out of 12, better twice, and worse twice.

Therefore the answer to RQ1 is definitely positive: both the considered simple functional size measures (namely, SFPs and tSFPs) provide effort estimates that are as accurate as those provided by IFPUG UFPs.

Note that it is also possible to split results according to project types: UFPs appear preferable to SFPs for new developments, while, for all the other combinations of project

type and size, there is hardly any difference in the performances of the considered functional size metrics.

5.2. Answer to RQ2

Research question RQ2 asks if UFPs and simple functional metrics (namely, SFPs and tSFPs) support effort estimation at significantly different levels of accuracy for projects that have relatively high (respectively, low) complexity.

Table 44 summarizes the results illustrated in Section 4.3.2 that are relevant for RQ2.

Table 44. Summary of results when complexity is considered.

Project Type	Subset	Complexity	Productivity	UFP vs. SFP	UFP vs. tSFP
New dev.	All	low	mean	<	=
New dev.	All	mid	mean	=	<
New dev.	All	high	mean	>	~>
New dev.	All	low	median	=	>
New dev.	All	mid	median	=	=
New dev.	All	high	median	=	=
New dev.	"Not too small"	low	mean	<	=
New dev.	"Not too small"	mid	mean	=	=
New dev.	"Not too small"	high	mean	>	~>
New dev.	"Not too small"	low	median	~>	=
New dev.	"Not too small"	mid	median	=	=
New dev.	"Not too small"	high	median	~>	=
Extensions	All	low	mean	<	=
Extensions	All	mid	mean	=	=
Extensions	All	high	mean	=	<
Extensions	All	low	median	=	=
Extensions	All	mid	median	=	=
Extensions	All	high	median	=	=
Extensions	"Not too small"	low	mean	=	=
Extensions	"Not too small"	mid	mean	=	=
Extensions	"Not too small"	high	mean	=	=
Extensions	"Not too small"	low	median	=	=
Extensions	"Not too small"	mid	median	=	=
Extensions	"Not too small"	high	median	=	=
Enhancements	All	low	mean	<	<
Enhancements	All	mid	mean	=	<
Enhancements	All	high	mean	>	=
Enhancements	All	low	median	~>	~>
Enhancements	All	mid	median	=	=
Enhancements	All	high	median	=	~>
Enhancements	"Not too small"	low	mean	<	=
Enhancements	"Not too small"	mid	mean	=	=
Enhancements	"Not too small"	high	mean	>	=
Enhancements	"Not too small"	low	median	=	>
Enhancements	"Not too small"	mid	median	=	=
Enhancements	"Not too small"	high	median	=	=

As a first observation, it is clear that the "=" sign is dominant in Table 44, as it was in Table 43.

When considering high-complexity projects, UFPs appear preferable to SFPs when mean productivity is used, but mainly equivalent when median productivity is used. In all the other cases, UFPs and simple functional metrics are either equivalent, or the best metric depends on the combination of project type, size, and productivity computation method. In all these cases, there is no apparent relationship between complexity and the most accurate metric.

Therefore, the answer to RQ2 is that UFPs and the considered simple functional size metrics SFPs and tSFPs support effort estimation at equivalent accuracy levels for

projects that have relatively high (respectively, low) complexity, with the only exception being that UFPs appear preferable to SFPs when mean productivity is used to estimate high-complexity projects.

5.3. Further Observations

According to our analysis, both the mean and median errors increase with complexity. This observation applies independently from project type and size, the way productivity is computed, and the functional size metric.

This seems to indicate that UFPs, as well as the simplified functional size measurement methods, fail to accurately represent the complexity of software projects, even when complexity is conceived in the same terms as in FPA.

6. Threats to Validity

A typical concern for considering only empirical data is the lack of a theoretical point of view, for example, in defining complexity and complex software projects. However, we started from some consolidated empirical evidence and practices about the criteria of software functional size, and we followed the common praxis of the community. One of the reasons why our results challenge a belief in the community (the more the UFP measure is “complex”, the better it is correlated to effort) probably comes from insufficient theoretical reflections. However, this is a generalized problem that our paper challenges.

Some decisions made while carrying out this study might have influenced the results. However, such decisions were necessary to perform the analysis. When dealing with the choices that most obviously could affect our results, we carried out some sensitivity analyses. For instance, concerning the criteria used to identify “not too small” projects when the median productivity model is used, we tried increasing (up to doubling) the minimum effort threshold that qualifies a project as “not too small” and we noticed no differences.

Another major concern in these kinds of studies is the generalizability of results outside the scope and context of the analyzed dataset. The ISBSG dataset is deemed the standard benchmark among the community, and it includes data from several application domains. Therefore, our results should be representative of a fairly comprehensive situation. However, additional studies could increase the generalizability of the results presented above.

Non-Applicability

In this paper, we addressed a very specific issue, which is relevant when IFPUG Function Points are used; one could wonder if they are applicable to other functional size measurement methods.

Some functional size measurement methods, like the COSMIC method, for instance [20], do not take complexity into consideration at all. Hence, our results do not apply to those methods.

Other estimation methods assign fixed weight to transactions and data. For instance, the ‘NESMA estimated’ method [21,22] assumes that all transactions are of medium complexity while all logic files are of low complexity; it has been shown that this leads to underestimating the size in general [23], and hence, when the NESMA estimated method is used, underestimation is bound to get worse for more complex products (i.e., products whose transactions and data have greater-than-average complexity). Our study is not necessary to draw this conclusion, which descends from the very definition of the method.

7. Related Work

Since the introduction of Function Point analysis, many researchers and practitioners have strived to develop simplified versions of the FP measurement process, both to reduce the cost and duration of the measurement process and to make it applicable when full-fledged requirement specifications are not yet available [3,24–31].

These simplified measurement methods were then evaluated with respect to their ability to support accurate effort estimation [5,21,32–39].

Lavazza et al. considered using only the number of transactions to estimate effort [6]: it was found that effort models based on the number of transactions appear marginally less accurate than models based on standard IFPUG Function Points for new development projects and marginally more accurate for projects extending previously developed software.

To the best of our knowledge, no studies considered classifying projects according to degrees of complexity (the notion of complexity being evaluated according to IFPUG Function Point Analysis criteria).

Since the 1990s, the early estimation of software size was achieved with different methods, like regression-like methods [40] or the “Early & Quick Function Point” (EQFP) method [41], which uses analogy to discover similarities between new and previously measured pieces of software and analysis to provide weights for software objects. Statistical estimation methods were first introduced by Lavazza et al., who studied the relationships between base functional components and size measures expressed in FPs [42].

More recently, machine learning methods have been used for software effort estimation. Some studies used natural language processing techniques to automatically extract FP phrases based on events from unstructured requirements documents in order to acquire transaction types without manual intervention. They used transformer-based semantic engines by a combination of architectures such as the BERT-BiLSTM-CRF ones [43]. Case-based reasoning and genetic algorithms were exploited with benchmark datasets and improved the accuracy of effort estimation [44].

Other studies dealt with effort estimation in agile development processes. Butt et al. proposed an estimation technique based on different categorizations of projects according to user story complexities and the developer’s expertise [45]. Another study proposed a comparison among user story points, use case points, IFPUG Function Points, and COSMIC Function Points in the agile domain by concluding that the COSMIC Function Points seemed to yield the best accuracy performance [46].

Some other very recent approaches proposed changes to the Function Point measurement procedures to obtain measures that support more accurate effort estimation. Hai et al. proposed a new algorithm to improve the weighting of transaction and data functions with respect to the IFPUG standard weights; their technique is based on the machine learning technique of Bayesian ridge regression, and on subsequent voting regressor mechanisms used for optimization purposes and based on ensemble learning methods such as random forest, neural networks, and lasso regressors [47]. They used the ISBSG dataset and claimed that the proposed algorithms improve effort estimation accuracy against the baseline method. Hoc et al. proposed an approach to improve software development estimation based on unadjusted Function Points and the value adjustment factor: they used log–log transformation and the Adam optimizer [48]. Hoc et al. show that some improvements on the error minimization side are yielded in the ISBSG dataset when compared to traditional methods such as, for example, mean effort. However, these latter studies do not achieve the relevancy and accuracy levels necessary to change traditional computations.

Those methods being far away from our study, we will not go further in detail into this research. It is worth noticing that these approaches are complementary to the one adopted in this paper, either for comparing their performance to other measures than UFPs, or focusing on task automation only, or using other families of machine learning techniques with respect to ours, or using the same techniques for the ambitious purpose of changing FPA, rather than just evaluating it, or for considering the agile domain, where also qualitative assessment is proposed for effort estimation.

8. Conclusions

Simplified functional size measures ignore the “complexity” of transactions, which is instead accounted for by traditional Function Point analysis. Some believe that this type of omission makes simplified measures less suitable for effort estimation, especially when

relatively complex software products are involved. To assess the truth of this belief, an empirical study was conducted, based on the analysis of the data from the ISBSG dataset.

Our analysis shows that UFPs do not appear to support more accurate effort estimation when performances over an entire dataset are considered.

When splitting the given dataset according to transaction complexity, UFP-based estimates appear more accurate than SFPs when the mean productivity is used in the estimation process. However, when the median productivity is used, UFPs and SFPs support equally accurate estimation. In addition, the transactional part of SFPs appears to generally support estimation at the same accuracy level as UFPs.

To sum up, the belief that, when considering “complex” projects, i.e., projects that involve many complex transactions and data, traditional Function Point measures support more accurate estimates than simpler functional size measures that do not account for greater-than-average complexity, is not confirmed.

An important by-product of the study is the observation that the accuracy of effort estimation decreases for increasing complexity, independently from the project type and size, the way productivity is computed, and the functional size metric. In other words, the complexity of software (as measured via FPA concepts) seems to affect effort estimation accuracy, but neither UFPs nor the simplified functional size metrics appear able to account for such complexity.

Future Work

Based on the observation above, how to effectively involve the notion of complexity in effort models is an interesting topic for future work. Namely, IFPUG Function Points are currently used for effort estimation via models that use as input the functional size measure, possibly together with other parameters representing characteristics of the product (e.g., nonfunctional requirements) the process, the developers involved, etc., i.e., via models of this type:

$$EstimatedEffort = f(FunctionPointSize, \langle ProductFeatures \rangle, \langle ProcessFeatures \rangle, \dots)$$

Our study showed that incorporating some notion of complexity in *FunctionPointSize* does not help, while complexity actually affects effort since more complex projects are estimated with larger errors by the techniques used in our work. Therefore, it seems a good to remove the notion of complexity from the measure of functional size (as carried out in tSFPs, for instance), measuring complexity properly (in a way still to be investigated) and using such complexity measures as additional parameters of the effort estimation models. The resulting model would be of the following kind:

$$EstimatedEffort = f(SimpleSizeMeasure, Complexity, \langle ProductFeatures \rangle, \langle ProcessFeatures \rangle, \dots)$$

It is expected that re-introducing complexity as a stand-alone, well-defined measure will improve effort estimates.

Author Contributions: Conceptualization, L.L. and R.M.; methodology, L.L., A.L. and R.M.; software, L.L. and A.L.; analysis, L.L.; writing—original draft preparation, L.L.; writing—review and editing, A.L. and R.M. All authors have read and agreed to the published version of the manuscript.

Funding: This research was partly supported by the “Fondo di Ricerca d’Ateneo” of the Università degli Studi dell’Insubria.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data we used are property of ISBSG; hence, we cannot provide them. However, the data can be requested from ISBSG <https://www.isbsg.org>, (accessed on 25 October 2024).

Conflicts of Interest: Author Roberto Meli was employed by the company Data Processing Organization Srl. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

Abbreviations

The following abbreviations are used in this manuscript:

EI	External Input
EIF	External Interface File
ENH	Enhancement
EO	External Output
EP	Elementary Process
EQ	External Inquiry
EQFPs	Early & Quick Function Points
EXT	Extension
FP	Function Point
FPA	Function Point Analysis
IFPUG	International Function Point User Group
ILF	Internal Logic File
ISO	International Standardization Organization
LF	Logic File
NEWs	new developments
PHs	PersonHours
SFP	Simple Function Point (as standardized by IFPUG)
SiFP	Simple Function Point (original definition)
tSFP	transactional Simple Function Point
UFP	Unadjusted Function Point

References

1. Albrecht, A.J. Measuring application development productivity. In Proceedings of the Joint SHARE/GUIDE/IBM Application Development Symposium, Monterey, CA, USA, 14–17 October 1979; Volume 10, pp. 83–92.
2. International Function Point Users Group (IFPUG). *Simple Function Point (SFP) Counting Practices Manual Release v2.1*; International Function Point Users Group (IFPUG): Princeton, NJ, USA, 2022.
3. Meli, R. Simple function point: A new functional size measurement method fully compliant with IFPUG 4. x. In Proceedings of the Software Measurement European Forum, Rome, Italy, 9–10 June 2011; pp. 145–152.
4. ISO/IEC 20926: 2003; Software Engineering “IFPUG 4.1 Unadjusted Functional Size Measurement Method” Counting Practices Manual. International Standardization Organization (ISO): Geneva, Switzerland, 2003.
5. Lavazza, L.; Meli, R. An evaluation of simple function point as a replacement of IFPUG function point. In Proceedings of the 2014 Joint Conference of the International Workshop on Software Measurement and the International Conference on Software Process and Product Measurement (IWSM-MENSURA), Rotterdam, The Netherlands, 6–8 October 2014; pp. 196–206.
6. Lavazza, L.; Liu, G.; Meli, R. Using Extremely Simplified Functional Size Measures for Effort Estimation: An Empirical Study. In Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), Bari, Italy, 5–7 October 2020; pp. 1–9.
7. Lavazza, L.; Locoro, A.; Meli, R. Using Machine Learning and Simplified Functional Measures to Estimate Software Development Effort. *IEEE Access* **2024**, *12*, 142505–142523. [[CrossRef](#)]
8. International Software Benchmarking Standards Group. *Worldwide Software Development: The Benchmark*; Release April 2019; International Software Benchmarking Standards Group: Melbourne, VIC, Australia, 2019.
9. Lavazza, L.; Locoro, A.; Meli, R. Software development effort estimation using function points and simpler functional measures: A comparison. In Proceedings of the 2023 Joint Conference of the International Workshop on Software Measurement and the International Conference on Software Process and Product Measurement (IWSM-MENSURA), Rome, Italy, 14–15 September 2023.
10. Fischman, L.; McRitchie, K.; Galorath, D.D. Inside SEER-SEM. *CrossTalk* **2005**, *18*, 26–28.
11. Hacaloglu, T.; Demirörs, O. Challenges of Using Software Size in Agile Software Development: A Systematic Literature Review. In Proceedings of the IWSM-Mensura, Beijing, China, 19–20 September 2018.
12. International Function Point Users Group (IFPUG). *Function Point Counting Practices Manual, Release 4.3.1*; International Function Point Users Group (IFPUG): Princeton, NJ, USA, 2010.
13. Lavazza, L. On the Effort Required by Function Point Measurement Phases. *Int. J. Adv. Softw.* **2017**, *10*, 107–120.
14. ISO/IEC 14143; Information Technology-Software Measurement-Functional Size Measurement. International Standardization Organization: Geneva, Switzerland, 2012.

15. González-Ladrón-de Guevara, F.; Fernández-Diego, M.; Lokan, C. The usage of ISBSG data fields in software effort estimation: A systematic mapping study. *J. Syst. Softw.* **2016**, *113*, 188–215. [\[CrossRef\]](#)
16. Boehm, B.W. Improving software productivity. *Computer* **1987**, *20*, 43–57. [\[CrossRef\]](#)
17. Shepperd, M.; MacDonell, S. Evaluating prediction systems in software project estimation. *Inf. Softw. Technol.* **2012**, *54*, 820–827. [\[CrossRef\]](#)
18. Kitchenham, B. Counterpoint: The problem with function points. *IEEE Softw.* **1997**, *14*, 29. [\[CrossRef\]](#)
19. Fenton, N.; Bieman, J. *Software Metrics: A Rigorous and Practical Approach*; CRC Press: Boca Raton, FL, USA, 2014.
20. COSMIC. COSMIC Measurement Manual for ISO 19761, Version 5.0. 2021. Available online: <https://cosmic-sizing.org/measurement-manual/> (accessed on 25 October 2024).
21. van Heeringen, H.; van Gorp, E.; Prins, T. Functional size measurement-Accuracy versus costs-Is it really worth it? In Proceedings of the Software Measurement European Forum (SMEF 2009), Rome, Italy, 27–28 May 2009.
22. Timp, A. uTip-Early Function Point Analysis and Consistent Cost Estimating. In *uTip # 03*; (Version # 1.0 2015/07/01); IFPUG: Princeton, NJ, USA, 2015.
23. Lavazza, L.; Liu, G. A Large-scale Empirical Evaluation of Function Points Estimation Methods. *Int. J. Adv. Softw.* **2020**, *13*, 182–193.
24. Horgan, G.; Khaddaj, S.; Forte, P. Construction of an FPA-type metric for early lifecycle estimation. *Inf. Softw. Technol.* **1998**, *40*, 409–415. [\[CrossRef\]](#)
25. Meli, R.; Santillo, L. Function point estimation methods: A comparative overview. In Proceedings of the FESMA, Amsterdam, The Netherlands, 8 October 1999; Citeseer: Forest Grove, OR, USA, 1999; Volume 99, pp. 6–8.
26. NESMA—the Netherlands Software Metrics Association. *Definitions and Counting Guidelines for the Application of Function Point Analysis. NESMA Functional Size Measurement Method Compliant to ISO/IEC 24570 Version 2.1*; NESMA—the Netherlands Software Metrics Association: Amsterdam, The Netherlands, 2004.
27. ISO/IEC 24570:2005; Software Engineering—NESMA Functional Size Measurement Method Version 2.1—Definitions and Counting Guidelines for the Application of Function Point Analysis. International Standards Organisation: Geneva, Switzerland, 2005.
28. Bernstein, L.; Yuhas, C.M. *Trustworthy Systems Through Quantitative Software Engineering*; John Wiley & Sons: Hoboken, NJ, USA, 2005; Volume 1.
29. Santillo, L.; Conte, M.; Meli, R. Early & Quick Function Point: Sizing more with less. In Proceedings of the 11th IEEE International Software Metrics Symposium (METRICS'05), Como, Italy, 19–22 September 2005; p. 41.
30. Iorio, T.; Meli, R.; Perna, F. Early & Quick Function Points® v3. 0: Enhancements for a Publicly Available Method. In Proceedings of the Proceedings Software Measurement European Forum (SMEF), Rome, Italy, 9–11 May 2007; pp. 179–198.
31. Lavazza, L.; Locoro, A.; Liu, G.; Meli, R. Estimating software functional size via machine learning. *ACM Trans. Softw. Eng. Methodol.* **2023**, *32*, 1–27. [\[CrossRef\]](#)
32. Wilkie, F.G.; McChesney, I.R.; Morrow, P.; Tuxworth, C.; Lester, N. The value of software sizing. *Inf. Softw. Technol.* **2011**, *53*, 1236–1249. [\[CrossRef\]](#)
33. Popović, J.; Bojić, D. A comparative evaluation of effort estimation methods in the software life cycle. *Comput. Sci. Inf. Syst.* **2012**, *9*, 455–484. [\[CrossRef\]](#)
34. Morrow, P.; Wilkie, F.G.; McChesney, I. Function point analysis using NESMA: Simplifying the sizing without simplifying the size. *Softw. Qual. J.* **2014**, *22*, 611–660. [\[CrossRef\]](#)
35. Lavazza, L.; Liu, G. An Empirical Evaluation of the Accuracy of NESMA Function Points Estimates. In Proceedings of the 14th International Conference on Software Engineering Advances (ICSEA 2019), Valencia, Spain, 24–28 November 2019; pp. 24–29.
36. Di Martino, S.; Ferrucci, F.; Gravino, C.; Sarro, F. Assessing the effectiveness of approximate functional sizing approaches for effort estimation. *Inf. Softw. Technol.* **2020**, *123*, 106308. [\[CrossRef\]](#)
37. Lavazza, L.; Liu, G. An Empirical Evaluation of Simplified Function Point Measurement Processes. *J. Adv. Softw.* **2013**, *6*, 1–13.
38. Meli, R. Early & Quick Function Point Method-An empirical validation experiment. In Proceedings of the International Conference on Advances and Trends in Software Engineering, Barcelona, Spain, 19–23 April 2015.
39. Ferrucci, F.; Gravino, C.; Lavazza, L. Simple function points for effort estimation: A further assessment. In Proceedings of the 31st Annual ACM Symposium on Applied Computing, Pisa, Italy, 4–8 April 2016; pp. 1428–1433.
40. Bock, D.B.; Klepper, R. FP-S: A simplified function point counting method. *J. Syst. Softw.* **1992**, *18*, 245–254. [\[CrossRef\]](#)
41. DPO. *Early & Quick Function Points Reference Manual—IFPUG Version*; Technical Report EQ&FP-IFPUG-31-RM-11-EN-P; DPO: Roma, Italy, 2012.
42. Lavazza, L.; Morasca, S.; Robiolo, G. Towards a simplified definition of Function Points. *Inf. Softw. Technol.* **2013**, *55*, 1796–1809. [\[CrossRef\]](#)
43. Han, D.; Gu, X.; Zheng, C.; Li, G. Research on Structured Extraction Method for Function Points Based on Event Extraction. *Electronics* **2022**, *11*, 3117. [\[CrossRef\]](#)
44. Hameed, S.; Elsheikh, Y.; Azzeh, M. An optimized case-based software project effort estimation using genetic algorithm. *Inf. Softw. Technol.* **2023**, *153*, 107088. [\[CrossRef\]](#)
45. Butt, S.A.; Ercan, T.; Binsawad, M.; Ariza-Colpas, P.P.; Diaz-Martinez, J.; Pineres-Espitia, G.; De-La-Hoz-Franco, E.; Melo, M.A.P.; Ortega, R.M.; De-La-Hoz-Hernandez, J.D. Prediction based cost estimation technique in agile development. *Adv. Eng. Softw.* **2023**, *175*, 103329. [\[CrossRef\]](#)

46. Ugalde, F.; Quesada-López, C.; Martínez, A.; Jenkins, M. A comparative study on measuring software functional size to support effort estimation in agile. In Proceedings of the CIbSE, Online, 6–9 May 2020; pp. 208–221.
47. Hai, V.V.; Nhung, H.L.T.K.; Prokopova, Z.; Silhavy, R.; Silhavy, P. A New Approach to Calibrating Functional Complexity Weight in Software Development Effort Estimation. *Computers* **2022**, *11*, 15. [[CrossRef](#)]
48. Huynh Thai, H.; Vo Van, H.; Ho, L.T.K.N. An approach to adjust effort estimation of function point analysis. *Lect. Notes Netw. Syst.* **2021**, *230*, 522–537.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.